

BABEŞ–BOLYAI TUDOMÁNYEGYETEM KOLOZSVÁR
MATEMATIKA ÉS INFORMATIKA KAR
INFORMATIKA SZAK

Szakdolgozat

Szimbolikus zene generálás mélytanulással



TÉMAVEZETŐ:

DR. CSATÓ LEHEL,
EGYETEMI TANÁR

SZERZŐ:

DÉGI NÁNDOR

2026

BABEȘ–BOLYAI UNIVERSITY OF CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND INFORMATICS
SPECIALIZATION: COMPUTER SCIENCE

Diploma Thesis

Generating symbolic music with deep learning



ADVISOR:

LEHEL CSATÓ, PHD.
PROFESSOR

AUTHOR:

NÁNDOR DÉGI

2026

UNIVERSITATEA BABEȘ–BOLYAI, CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA INFORMATICĂ

Lucrare de licență

Generarea muzicii cu învățare profundă



CONDUCĂTOR ȘTIINȚIFIC:
PROFESOR DR. LEHEL CSATÓ

ABSOLVENT:
NÁNDOR DÉGI

2026

BABEȘ–BOLYAI UNIVERSITY OF CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND INFORMATICS
SPECIALIZATION: COMPUTER SCIENCE

Diploma Thesis

Generating symbolic music with deep learning

Abstract

This thesis investigates the application of deep learning methods in symbolic music generation, with particular focus on the integration of Markov chains, variational autoencoders (VAE), and transformer models. The research centers on the Zha system, which applies a hybrid architecture to model the musical creative process.

The thesis provides detailed analysis of the theoretical foundations and practical implementations of each model. We present the melody generation capabilities of Markov chains, creative variation techniques in VAEs, and polyphonic structure addition in Transformer models. Special attention is given to structured musical generation, ensuring long-term coherence, and computational efficiency.

The Zha system's innovative approach combines Markov-based melody generation with VAE-driven variations and Transformer-enhanced harmonic structures. Empirical evaluation shows that the hybrid approach significantly improves the quality and diversity of generated music compared to traditional single-model approaches.

This research contributes to the field of computational musical creativity by demonstrating the effective integration of different deep learning paradigms and solving complex modeling challenges of musical structure.

This work is the result of my own activity. I have neither given nor received unauthorized assistance on this work.

2026

NÁNDOR DÉGI

ADVISOR:
LEHEL CSATÓ, PHD.
PROFESSOR

Tartalomjegyzék

1. Bevezetés	1
1.1. Motiváció	1
1.2. Kutatási célok	2
2. Háttér és elméleti alapok	3
2.1. Szimbolikus zenegenerálás	3
2.1.1. Piano-roll reprezentációk	4
2.1.2. Tokenizált szekvencia-reprezentációk	4
2.1.3. Reprezentációs kihívások	5
2.2. A rendszer architektúráis alapjai	6
2.2.1. Többmodelles generálási stratégia	6
2.2.2. Adatfeldolgozási pipeline	6
2.3. Értékelési mérőszámok	7
2.3.1. Perplexitás	7
2.3.2. Hangmagasság-entrópia	7
2.3.3. Ritmikai konzisztencia	8
3. Markov láncok dallamgeneráláshoz	9
3.1. Szerepe	9
3.2. Elméleti alapok	9
3.2.1. A Markov-tulajdonság zenei alkalmazása	9
3.2.2. Magasabb rendű Markov-modellek	10
3.2.3. Laplace add- α simítás	10
3.3. Optimalizált algoritmusok	11
3.3.1. Vektorizált számítások	11
3.3.2. GPU-gyorsítás	11
3.4. Zenei predikció és generálás	11
3.4.1. Skálatudatos generálás	11
3.4.2. Intervallum-alapú reprezentáció	12
3.4.3. Hőmérséklet-vezérelt mintavételezés és top-K szűrés	12
3.4.4. Ismétlés-büntetés	13
3.4.5. Határjelölő szimbólumok	13
3.5. Implementációs részletek	13
3.5.1. Markov-lánc implementáció	13

TARTALOMJEGYZÉK

3.5.2.	Rejtett Markov-modell kiegészítés	14
3.5.3.	Tanítási folyamat	15
3.6.	Korlátozások	15
3.7.	Összefoglalás	15
4.	Variációs autoenkóderek csoport-orbitális konzisztenciával	16
4.1.	Bevezetés	16
4.2.	Csoport-orbitális konzisztencia	17
4.2.1.	GOLC módszer	17
4.3.	Implementáció és architektúra	18
4.3.1.	Kódoló és dekódoló	18
4.3.2.	Orbitális konzisztencia számítása	18
4.4.	Elméleti tulajdonságok	19
4.4.1.	Állítás 1: Invariancia feltétele	19
4.4.2.	Állítás 2: Diagonális posterior-stabilitás	19
4.4.3.	Állítás 3: Tanulási stabilitás	19
4.5.	Empirikus megfigyelések	20
4.6.	Implementációs részletek	20
4.6.1.	Tanítási konfiguráció	20
4.6.2.	Tanítási hurok	21
4.7.	Kapcsolat korábbi munkákhoz	21
4.8.	Összefoglalás	22
5.	Transformer hálózatok zenegeneráláshoz	24
5.1.	Bevezetés	24
5.2.	A Figyelem mechanizmusa	25
5.2.1.	Önfigyelési mechanizmus	25
5.2.2.	Többfejű figyelem	25
5.3.	A Zha Transformer architektúra	26
5.3.1.	Pozíciókódolás	26
5.3.2.	Multitrack kiterjesztés	27
5.3.3.	Akkord- és tempó-conditioning	27
5.3.4.	Szekció-alapú memória	28
5.4.	Mintavételezés és generálás	28
5.5.	Tanítás	28
5.6.	Teljesítmény és komplexitás	29
5.6.1.	Számítási komplexitás	29
5.6.2.	Memória menedzsment	30
5.7.	Összefoglalás	30
6.	Tanítási és optimalizálási technikák	32
6.1.	Modellspecifikus tanítási paradigmák	32

TARTALOMJEGYZÉK

6.1.1.	Variációs autoenkóderek	32
6.1.2.	Transformer	33
6.2.	Közös tanítási infrastruktúra	33
6.2.1.	Optimalizáló és precízió	33
6.2.2.	Korai leállás és checkpoint mentés	34
6.2.3.	Adatkezelés és LRU cache	34
6.2.4.	Gradiens-akkumuláció és clipping	34
6.2.5.	Heartbeat logging	34
6.2.6.	Hardver	35
6.3.	HuggingFace MidiCaps streaming	35
6.4.	Optimális konfigurációk	35
6.5.	Tanítási monitorozás	36
7.	A rendszer tervezése	37
7.1.	A rendszer magas szintű áttekintése	37
7.2.	Adatfolyam és modell integráció	38
7.2.1.	Szekvenciális feldolgozási lánc architektúra	38
7.2.2.	Kombinált generálási algoritmus	38
7.2.3.	Skálaszűrés és regiszter-korlátozások	39
7.2.4.	Adatfolyam részletei	40
7.3.	Adat-előfeldolgozási folyamat	40
7.3.1.	MIDI elemzés és sávok szétválasztása	40
7.3.2.	Kvantálás	41
7.3.3.	Tokenizációs séma	42
7.3.4.	Adathalmaz statisztikái	42
7.4.	Modellarchitektúrák	43
7.5.	Tanítási módszertanok	43
7.6.	Inferenciás motor	44
7.6.1.	Végponttól-végpontig FastAPI integráció	44
7.7.	Értékelés és kísérletek	44
7.8.	Összefoglalás	45
8.	Következtetések és jövőbeli irányok	46
8.1.	Összegzés	46
8.1.1.	Hozzájárulások	46
8.1.2.	Gyakorlati eredmények	47
8.2.	Korlátok és kihívások	48
8.2.1.	Technikai korlátok	48
8.2.2.	Zenei korlátok	48
8.3.	Jövőbeli kutatási irányok	49
8.3.1.	Rövid távú, közvetlenül következő munka	49

TARTALOMJEGYZÉK

8.3.2.	Hosszabb távú irányok	49
8.4.	Záró megjegyzés	50
9.	Kiegészítő bizonyítások	51
9.1.	Konzisztencia veszteség a zenei folytonosságért	51
9.1.1.	Matematikai definíció	51
9.1.2.	Integrálás a teljes veszteségfüggvénybe	51
9.1.3.	Gradiens levezetés	52
9.2.	Szekció-alapú memória és átmenet-simítás	52
9.2.1.	Memória reprezentáció	52
9.2.2.	Átmeneti simítás	53
9.2.3.	Komplexitás	53
9.3.	HMM integráció zenei jellemzőkkel	53
9.3.1.	Jellemzővektor konstrukció	53
9.3.2.	Jellemzők definíciója	54
9.3.3.	HMM emissziós valószínűségek jellemzőkkel	54
9.4.	Ritka mátrix optimalizálás Markov láncokhoz	54
9.4.1.	Memória komplexitás	54
9.4.2.	Ritka reprezentáció	55
9.4.3.	Hozzáférési komplexitás	55
9.5.	Intervallum-alapú transzpozíció	55
9.5.1.	Transzformáció	55
9.5.2.	Transzpozíció invariancia	55
9.5.3.	Korlátos intervallum tér	56
9.6.	GPU-alapú batch normalizálás	56
9.6.1.	Stabil normalizálás	56
9.6.2.	Vektorizált implementáció	56
9.6.3.	Komplexitás analízis	56

Ábrák jegyzéke

4.1.	A VAE architektúra általános felépítése a kódoló és dekódoló rétegekkel, valamint a látens tér reprezentációval.	17
4.2.	Reziduális blokk normalizációval és skip connectionnel.	18
4.3.	A látens tér KL divergenciájának eloszlása dimenziók szerint.	23
4.4.	A reparametrizációs hőmérséklet paraméter hatása a generálás diverzitására. . .	23
5.1.	A Transformer architektúra felépítése többfejű figyelem rétegekkel, pozíciókódolással és előreccsatoló hálózattal.	25
5.2.	Attention súlyok vizualizációja különböző fejekben.	26
5.3.	Különböző sampling stratégiák (greedy, temperature, top-k, nucleus) hatása. . .	29
5.4.	Perplexitás-analízis különböző szekvenciahosszaknál.	30
5.5.	Tanítási görbék: veszteség és perplexitás alakulása az epoch-ok során.	30
7.1.	Végponttól végpontig tartó Zha folyamat: Markov $\rightarrow$ VAE $\rightarrow$ Transformer $\rightarrow$ MIDI, mutatva a három modell integrációját és a feldolgozási láncot.	37
7.2.	Strukturált generálási folyamatára	39
7.3.	MIDI előfeldolgozási lánc: nyers MIDI fájlból jelsorba alakított szekvenciáig, beleértve a többsávós szétválasztást, kvantálást és jelsor-képzést.	40

Táblázatok jegyzéke

6.1.	A Zha trénerek konkrét konfigurációja a kód defaultjai szerint.	35
7.1.	Lakh MIDI Dataset Clean statisztikák (Kaggle). A felhasznált részhalmaz tartalmazza a megfelelő hosszúságú ($256 \leq \text{tokens} \leq 4096$) és minőségű MIDI fájlokat.	42
7.2.	Sávszétválasztási statisztikák a 17,526 fájlos dataset előfeldolgozás után	42

1. fejezet

Bevezetés

1.1. Motiváció

Az algoritmikus zeneszerzés régi terület: Hiller és Isaacson [Hiller Jr. és Isaacson, 1958]-ja, ahol egy IBM gépen sztochasztikus szabályrendszer írta meg a négy tételt. Évtizedeken át a kérdés úgy szólt: *milyen szabályrendszer* írja le a tonális zenét. A mélytanulás megjelenésével a kérdés átalakult: ahelyett, hogy formális zeneelméleti szabályokat kódolnánk, *milyen mintázatok* talál egy hálózat egy meglévő zenei korpuszban; a terület átfogó áttekintéseit lásd [Briot et al., 2017] és [Ji et al., 2023] munkáiban. Ez a dolgozat a második szempontra koncentrált, méghozzá szimbolikus (MIDI) reprezentáció, ahol az események diszkrét és pontosan megcímkézhetők.

A pusztán mintázattanulás azonban önmagában nem elég, mert egyetlen modell sem tudja egyszerre lefedni a zenegenerálás összes aspektusát: a lokális dallami kontúr, a transzpozíció-invariáns stílust és a hosszabb távú szerkezetet. Ez a tapasztalat motiválta a rendszer tervezését, amelyben három különböző modellt osztok szét három különböző zenei aspektus között, és nem egyetlen modelltől próbálok mindent kihozni.

Ebben a kutatásban azt vizsgálom, hogyan lehet három különböző adaptív módszert egymás mellé tenni úgy, hogy mindegyik a zenei alkotás más oldalát kezelje. A Markov-láncot (rejtett Markov-modell kiterjesztéssel) a helyi, skálához kötött dallami statisztikára használom: ez a rész be tudja olvasni a [Cuthbert és Ariza, 2010a]-en keresztül a hangnemet, és csak az ahhoz tartozó hangokból mintavételez. A VAE (és annak GOLC-kiterjesztése) egy 128-dimenziós pitch-eloszlás látens manipulációját végzi: stílusbeli „arcútváltást” itt lehet a legkönnyebben paraméterezni. A Transformer pedig a többsávú, polifón kíséretet (basszus, dobok, akkordok) generálja cross-attention koordinációval. Nem azt állítom, hogy ez az osztás egyedüli helyes

felbontás, csak hogy a három modellnek így van a legkevésbé átfedő felelőssége.

1.2. Kutatási célok

A dolgozat elsődleges célja a rendszer kidolgozása és elemzése: ez egy hibrid mélytanulási architektúra szimbolikus zenegenerálásra. Három egymással összefüggő kérdést vizsgálok:

Először is, hogyan érdemes felosztani a generálási feladatot három különböző modell között úgy, hogy mindegyik saját, jól körülhatárolt zenei aspektust kezeljen, és a felelősségek minél kevésbé legyenek átfedők. A Zha-ban a Markov-lánc a hangnemtudatos dallami alapot adja, a VAE (és annak GOLC-kiterjesztése) a transzpozíció-invariáns látens manipulációt, a Transformer pedig a polifón kíséret és többsávós struktúra szintézisét.

Másodszor, milyen gyakorlati implementációs kihívások merülnek fel a három modell egymás melletti tanításánál és inferenciájánál — a számítási költség, a tanítási stabilitás és a memóriakezelés szempontjából — és milyen tervezési döntésekkel kezelhetők. Harmadszor, milyen formában építhető be zeneelméleti tudás (skála, akkordfunkció, hangnem-felismerés) az adatvezérelt modellekbe anélkül, hogy túlzottan korlátozná a kreatív kimenetet.

A dolgozat ezeket a kérdéseket részletes modell-specifikus fejezetekben (Markov, VAE/GOLC-VAE, Transformer) és egy integrációs fejezetben tárgyalja, amely a pipeline egészét írja le. A munka egyaránt szolgál gyakorlati implementációs leírásként és kísérleti platformként, ahol a hibrid generatív megközelítés lehetőségeit és korlátait elemezni lehet.

2. fejezet

Háttér és elméleti alapok

2.1. Szimbolikus zenegenerálás

A zene ábrázolásánál két alapvető megközelítés közül választhatunk. A hangfájlok folytonos hullámformákban tárolják az audio jelet, ez pontosabb az emberi hallás szempontjából. Szimbolikus formában viszont diszkrét eseményeket rögzítünk: hang kezdete, időtartama, erőssége. Neurális modellek számára ez utóbbi alkalmasabb. A statisztikai mintaillesztés és a szekvencia-modellezés is egyszerűbbé válik, ha tokenekről vagy eseményekről, nem pedig folytonos jelről beszélünk.

Három fő reprezentációs stratégia létezik a szakirodalomban, bár egyik sem tökéletes. MIDI események időbélyeggel ellátott parancsokból állnak, NOTE_ON, NOTE_OFF, CONTROL_CHANGE. Minden esemény kódol hangmagasságot (0-127), velocity értéket (1-127), csatornát és időbélyeget. A formátum széleskörű kompatibilitása miatt a legtöbb munka (beleértve saját implementációmat is) ezt használja bemenetnek és kimenetnek.

A MIDI formátum azonban komoly gyakorlati problémákat vet fel. Az események szabálytalan időintervallumokban fordulnak elő. A neurális hálózatok, különösen a rekurrens modellek, fix hosszúságú batcheket várnak. Korai kísérleteim során ez batch padding és masking szempontjából bizonyult nehéznek. A dinamika 128 erősség értékre korlátozódik, ami egyébként is túl finomra sikerült az én adatkészletemhez képest (a legtöbb MIDI fájl alig használ 10-15 különböző szintet). A többszólamúság megjelenítése pedig valódi kompromisszum. Több csatornát használhatok párhuzamos hangfolyamokhoz, vagy összevonhatom őket egyetlen listába, ez utóbbi megközelítést választottam, bár látható, hogy összezavarja a Markov-alapú tanuló részeket.

2.1.1. Piano-roll reprezentációk

A piano-roll kétdimenziós rácsként ábrázolja a zenét: a sorok a hangmagasságokat, az oszlopok a fix időlépéseket (jellemzően tizenhatod hangokat) jelölik. Bináris vagy többszintű mátrix jelzi, mely hangok szólalnak meg az egyes időszeletekben. Konvolúciós hálózatok szempontjából előnyös, a szabályos rács természetes módon kódolja a polifóniát: minden oszlopban több hang lehet egyszerre, így a Transformer modellnél később nem kellett külön mechanizmust implementálni az egyidejű események kezelésére.

A piano-roll reprezentáció ugyanakkor memória-intenzív. Több perces MIDI darabokon nagy, ritka mátrixok keletkeznek; az RTX 3090 24 GB VRAM-jával is korlátozni kellett a batch méretet 16-ra a VAE tanításánál, hogy ne kerüljön a memóriahatárra. A rögzített, tizenhatod hangos időlépések elvesztik a swing- és rubato-árnyalatokat. Ez a Zha esetében elfogadható kompromisszum, mert a tanításhoz használt korpuszok amúgy is kvantáltak, ám felveti a kérdést, vajon a kvantált kimenet zeneileg autentikus-e. A dinamikai információt külön csatornában kellene kódolni, ami tovább növeli a dimenzionalitást — ez volt az egyik fő ok, amiért a Transformer modulban végül a tokenizált formátumhoz tértem vissza.

2.1.2. Tokenizált szekvencia-reprezentációk

Natural Language Processing-ből inspirálódva számos rendszer tokenizált szekvenciákat használ. Minden zenei esemény (NOTE_ON_60, TIME_SHIFT_4, VELOCITY_80 stb.) önálló tokenné válik. A REMI-stílusú kódolás [Huang és Yang, 2020] egyesíti a hangmagasságot, időtartamot és dinamikát egyetlen lineáris szekvenciában, amelyre a Transformer architektúrák közvetlenül alkalmazhatók; ez lett a Zha Transformer moduljának kiindulópontja is.

Ez a forma rugalmas egyensúlyt enged az időbeli felbontás és a szekvenciahossz között. A TIME_SHIFT granularitása finomhangolható; én végül a tizenhatod hangos felbontásnál maradtam, ami a használt korpusz tipikus ütempárjaihoz elegendő. A többszólamúságot speciális tokenekkel jelzem, bár 4–5 egyidejű hang fölött a generálás már kombinatorikusan robbanékonyá és instabillá válik.

2.1.3. Reprezentációs kihívások

A fenti kompromisszumok mellett három, egymással összefüggő kihívás marad minden megközelítésben: a dinamika kódolása, a polifónia modellezése és a kifejező időzítés visszaadása.

Dinamika kódolása A MIDI 128 sebességszintjét a szókincs méretének kordában tartására rendszerint kevesebb csoportra bontják; a Zha-ban 8 velocity-csoportot használok. Ez óhatatlanul feláldoz árnyalatokat, de a gyakorlatban ritkán volt szükség finomabb felbontásra. Folytonos beágyazásokat is kipróbáltam korai tesztekben, de a diszkrét, autoregresszív Transformer-mintavételbe integrálva bizonytalan eredményeket adtak.

Polifónia modellezése A valódi polifónia (több egyidejű, független hangfolyam) többsávos megjelenítést vagy egymásba ágyazott jelfolyamokat igényel. Az egyfolyamú kódolásnak az egyidejű eseményeket speciális tokenekkel vagy nulla idő-eltolásokkal kell jeleznie; a Zha Transformer tanítása során ezt nem mindig sikerült jól megtanulnia, ami rosszul összehangolt akkordokat eredményezett generáláskor. Ezt a problémát a multitrack ággal és a sávok közötti cross-attention koordinációval kerülöm meg.

Időzítés és kifejezőerő Az emberi előadások kifejező időzítést mutatnak (rubato, swing, mikro-ütemezési variációk), amit a fix rácsos kvantálás óhatatlanul eltüntet. Változó hosszúságú TIME_DELTA tokenek kísérleti bevezetését (a Music Transformer [Huang et al., 2018] mintájára) megpróbáltam, de a megnövekedett szekvenciahossz a modell tanulását instabillá tette. Az előadás-szintű árnyalatok és a kotta összehangolása túlmutat a végponttól végpontig generáló modellek hatókörén.

Összességében a szimbolikus zenei reprezentáció gondos tervezési döntéseket igényel a kifejezőképesség, a modell kezelhetősége és a tanítási adat ritkasága közötti egyensúly megtalálásához. A terület fejlődik a gazdagabb token-grammatikák, hierarchikus kódolások és gráf-alapú reprezentációk felé, de egyelőre egyik sem bizonyult univerzálisan jobbnak az egyszerűbb, REMI-alapú megközelítésnél.

2.2. A rendszer architektúrális alapjai

2.2.1. Többmodelles generálási stratégia

A rendszerben három különböző generatív modellt kötök össze egyetlen pipeline-ban. A felosztás célja, hogy mindegyik modell a zenei generálás más aspektusát kezelje, és minimális legyen a felelősség-átfedés. A Markov-lánc alapú komponens valószínűségi megközelítést használ, zeneelméleti szabályokkal (skála, akkordfunkció) kiegészítve. Ez a rész különösen jól kezel rövid távú mintázatokat és helyi harmóniai kapcsolatokat; engem kicsit meglepett, mennyire jól teljesít alapvető akkordprogresszióknál még az egyszerű elsőrendű Markov-lánc is.

Variációs Autoenkóderek (VAE-k) a rejtett tér manipulációjával lehetővé teszik a kreatív változatokat. Alacsony dimenziós reprezentáció lehetővé teszi sima átmeneteket és rendszeres változtatásokat, miközben megőrzi a zenei összhangot. Ez különösen hasznos stílusváltásnál vagy téma-variációk létrehozásánál, bár a latent space részletes feltérképezését nem végeztem el, ami egy lehetséges jövőbeli irány lehetne.

A Transformer komponens a hosszú távú szerkezeti tudatosságért felel. Önfigyelem mechanizmusa révén képes távoli zenei események közti kapcsolatokat modellezni anélkül, hogy a rekurrens hálózatok gradiens-eltűnési problémájával kellene megküzdenem. Kulcsfontosságúvá vált komplex zenei formák előállításánál, ahol a távoli szakaszok között szerkezeti kapcsolatok állnak fenn. Nyitva marad a kérdés, vajon a 2048-as maximális pozíciókódolás-hossz elegendő-e igazán hosszú formákhoz, mint a szonáta-forma — a gyakorlatban a pipeline-ban a generálási hosszak ennél jóval rövidebbek.

2.2.2. Adatfeldolgozási pipeline

MIDI fájlok feldolgozásakor az egyes hangszerek hangjait és metaadatait külön-külön elemzem. A hangmagasság-információkat súlyozottan rögzítem, a velocity értékek normalizált formában mutatják a dinamikai viszonyokat. Az implementáció a teljes MIDI fájlt egy 128-dimenziós hangmagasság-hisztogrammá alakítja, ami tömör, mégis informatív képet ad a harmonikus tartalomról.

A zeneelméleti integráció központi szerepet játszik. A hangnem-felismerést a `music21.analyze('key')` hívás végzi [Cuthbert és Ariza, 2010b]; a music21 ezt a Krumhansl-féle kulcsprofil-illesztésre építi [Krumhansl, 1990]. A skála-illesztés biztosítja,

2. FEJEZET: HÁTTÉR ÉS ELMÉLETI ALAPOK

hogy a generált hangok illeszkedjenek a kiválasztott hangnemhez, de a gyakorlatban a Markov-modell néha mégis ad ki skálán kívüli hangokat: a magasabb rendű intervallum-átmenetek felülírhatják a skálamaszkot, ami zeneileg gyakran érdekes kromatikus eseményeket eredményez. Az akkord-elemzés római szám jelölést használ, ami a funkcionális harmónia formális megjelenítését teszi lehetővé (pl. I-IV-V-I progressziók azonosítása).

2.3. Értékelési mérőszámok

A generált zene minőségének értékelése objektív és szubjektív módszereket egyaránt igényel. Az objektív statisztikai mérőszámok — perplexitás, hangmagasság-entrópia, ritmikai konzisztencia — automatikusan számszerűsíthetők, és reprodukálható összehasonlítást tesznek lehetővé. A szubjektív, hallgatói teszteken alapuló értékelés szintén indokolt lenne, de a jelen dolgozat keretein belül ezt nem végeztem el teljes mértékben.

2.3.1. Perplexitás

A nyelvmodellezésből származó perplexitás azt méri, mennyire bizonytalan a modell egy szekvencia következő elemének előrejelzésében. N hosszúságú tesztszekvencia esetén, ahol $P(s_t)$ a t -edik token valószínűsége:

$$\exp\left(-\frac{1}{N} \sum_t \log P(s_t)\right).$$

Szimbolikus zenében az alacsonyabb perplexitás a tanítási adatokhoz való szorosabb statisztikai illeszkedést jelez, de nem feltétlenül jár együtt jó zenei minőséggel: a modell túlilleszkedés mellett is alacsony perplexitást érhet el, miközben mechanikusan ismétlődő kimenetet ad — ezt a Markov-modell korai verzióinál magam is tapasztaltam. A perplexitás tehát hasznos diagnosztikai eszköz, de önmagában nem elegendő minőségi kritérium.

2.3.2. Hangmagasság-entrópia

A hangmagasság-entrópia számszerűsíti a generált eredmény hangmagasságainak változatosságát. Ha p_i jelöli az i -ik hangmagasság empirikus gyakoriságát, akkor az entrópia $H = -\sum_i p_i \log p_i$ formában adódik. A magas entrópia gazdag dallami változatosságra utal, de

2. FEJEZET: HÁTTÉR ÉS ELMÉLETI ALAPOK

a rendkívül magas értékek strukturálatlan, véletlenszerű zajra is utalhatnak. A generált minták hangmagasság-entrópiájának és a tanítókörpusz entrópiájának összehasonlítása felfedheti az alul- vagy túldiverzifikáltságot. Az entrópia kiszámítható csúszó ablakokon keresztül is, hogy értékeljük mind a helyi, mind a globális változatosságot. Fontos megjegyezni, hogy a hangmagasság-entrópia önmagában nem veszi figyelembe a szekvenciális struktúrát és a harmonikus kontextust, ezért más mérőszámokkal együtt kell használni.

2.3.3. Ritmikai konzisztencia

A ritmikai konzisztencia azt méri, mennyire illeszkedik a generált ritmus az emberhez hasonló időbeli mintákhoz és természetes ütemezési variációkhoz. Az inter-onset-intervallum (IOI) variancia és a szinkópációs indexek értékelik a szekvenciák időzítési szabályosságát és kifejező jellegét. Magas IOI szórás szabálytalan, kaotikus időzítésre utalhat, míg az extrém alacsony variancia túlságosan merev, mechanikusan kvantált eredményeket jelez. Egyes értékelési keretrendszerek a generált ritmikai jellemzők (mint például a swing arány vagy a szinkópációs sűrűség) korrelációját számítják ki referencia adatkészletekkel, így értékelve a természetességet és az autentikusságot.

3. fejezet

Markov láncok dallamgeneráláshoz

3.1. Szerepe

A Markov-lánc a Zha rendszer dallami komponense. Feladata, hogy egy adott hangnemben (és skálában) monofón dallamot generáljon: ez kerül a pipeline elejére, és erre építenek később a VAE és a Transformer modulok. A bonyolultabb modellektől ezt a komponenst két dolog különbözteti meg. Egyrészt nem szekvenciális neurális hálózat, így a tanítása egyszerű frekvencia-számolás, gyorsan futtatható és értelmezhető. Másrészt zeneelméleti tudás (skála, akkordfunkció, intervallum-korlátok) közvetlenül beilleszthető a mintavételezésbe, anélkül hogy újra kellene tanítani a modellt.

A modul két szempontból tér el a tankönyvi Markov-lánctól: egyrészt hierarchikus, magasabb rendű állapotmodellt használ a kontextus szélesítésére; másrészt egy `hmmlearn`-alapú rejtett Markov-moddellel egészül ki [Rabiner, 1989], amely a hang-emissziókat egy rejtett zenei állapot (például emelkedő vagy ereszkedő mozgás) feltételes kibocsátásaként modellezi.

3.2. Elméleti alapok

3.2.1. A Markov-tulajdonság zenei alkalmazása

Egy $\{X_t\}_{t \geq 0}$ diszkrét idejű sztochasztikus folyamat akkor rendelkezik a Markov-tulajdonsággal, ha a következő állapot csak az aktuális állapottól függ [Norris, 1998]:

$$P(X_{t+1} = s_j \mid X_t = s_i, X_{t-1} = s_k, \dots) = P(X_{t+1} = s_j \mid X_t = s_i) = p_{ij}.$$

Zenei kontextusban az állapotok MIDI hangmagasságokat képviselnek (0–127), és az át-

3. FEJEZET: MARKOV LÁNCOK DALLAMGENERÁLÁSHOZ

meneti valószínűségek a helyi dallami folytonosságot modellezik. A Zha implementáció a rendet konfigurálhatóan 2 és 6 között veszi fel (a kódban a felső korlát: `max_order = min(6, self.order)`), alapértelmezetten 3-mal. A magasabb rend hosszabb dallami kontextust enged, de exponenciálisan növeli a meg nem figyelt n -esek számát, ezért a rend választása mindig kompromisszum.

3.2.2. Magasabb rendű Markov-modellek

Az n -edrendű Markov-modell az előző n állapotra támaszkodik:

$$P(X_{t+1} \mid X_t, X_{t-1}, \dots, X_{t-n+1}).$$

Az implementációban ezeket a magasabb rendű átmeneteket szótár-alapú (sparse) tárolással valósítom meg, mert a 128-as alapállapot-térrel egy 6-rendű mátrix teljes alakban $128^7 \approx 5,6 \cdot 10^{14}$ cellát igényelne. A szótár csak a tényleges tanítási adatban előforduló n -eseket tartja meg.

3.2.3. Laplace add- α simítás

A ritka, magasabb rendű táblákban a számlálók többsége 0 vagy 1, így a soha meg nem figyelt folytatások a generálás során nulla valószínűségűek maradnak, és teljes egészében kizáródnak a mintavételezésből. Ennek enyhítésére Laplace add- α simítást alkalmazok az $\alpha \geq 0$ paraméterrel:

$$\hat{p}(s' \mid \mathbf{s}) = \frac{N(\mathbf{s}, s') + \alpha}{\sum_{s''} N(\mathbf{s}, s'') + \alpha \cdot |V|},$$

ahol $|V| = 128$ a MIDI hangmagasság-szókincs mérete és α a `MarkovChain` konstruktorának `laplace_alpha` paramétere. Az alapérték 0, ami a simítás nélküli, korábbi viselkedést őrzi meg. A 0,1 és 1,0 közötti értékek mellett a generálás az olyan ritka, csak egyszer-kétszer megfigyelt magasabb rendű kontextusokban is megőriz némi mintavételi diverzitást, amelyek egyébként determinisztikussá zsugorodnának.

3.3. Optimalizált algoritmusok

3.3.1. Vektorizált számítások

A hagyományos Markov-generálás soros valószínűség-számítással működik. A teljes 128-es átmeneti vektort egyetlen NumPy/PyTorch indexeléssel olvasom ki, és egyetlen `np.random.choice` hívással mintavételezek:

```
next_probs = transitions[current_state]
next_note = np.random.choice(128, p=next_probs / next_probs.sum())
```

3.3.2. GPU-gyorsítás

Nagy állapotterek esetén PyTorch CUDA tenzorokra is fel tudja venni az átmeneti mátrixot. A `CUDA0ptimizer` osztály (`markov_chain.py`) ezt a be- és kipakolást kezeli; a CuPy backend opcionális, és ha nem érhető el, az implementáció PyTorch tenzorokkal dolgozik tovább. Reprezentatív benchmarkot nem futtattam, ezért konkrét gyorsulási arányt nem közlök.

3.4. Zenei predikció és generálás

3.4.1. Skálatudatos generálás

A dallam mindig egy adott zenei skálához igazodik. A bemeneti MIDI fájl hangnemét a `music21.analyze('key')` hívás adja meg [Cuthbert és Ariza, 2010b]; a `music21` ezt a Krumhansl-féle kulcsprofil-illesztésre építi [Krumhansl, 1990], amely a 12 pitch class tonális hierarchia szerinti súlyait illeszti a bemenet hangmagasság-eloszlására. A kimenetből a `_get_scale_pitches` függvény bontja ki a megengedett pitch class halmazt. Ez a halmaz aztán maszkként szűri az átmeneti valószínűségeket:

```
scale_pitches = self._get_scale_pitches(key) # pl. C-dur:
              {0,2,4,5,7,9,11}
mask = np.isin(np.arange(128) % 12, list(scale_pitches))
3 filtered_probs = probs * mask
```

A gyakorlatban a Markov-lánc néha mégis skálán kívüli hangokat ad ki: ez akkor fordul elő, amikor a tanult magasabb rendű kontextus felülírja az egyszerű skálamaszkot (intervallum-alapú átmeneten keresztül). Ezt a kromatikus kilengést meghagytam, mert zeneileg gyakran érdekes átmeneti hangokat termel.

3.4.2. Intervallum-alapú reprezentáció

A pitch-alapú átmenetek mellett a modul intervallum-alapú átmeneteket is tanul: $i_j = p_{j+1} - p_j$, $i \in [-12, +12]$ félhang tartományra korlátozva. Ennek két előnye van: (i) transzpozíció-invariáns (egy más hangnembe transzponált dallam ugyanazokat az intervallumokat adja, lásd a 9.5. függelék), és (ii) a tárhelyigény $|S|^2 = 128^2$ -ről $|S| \cdot 25$ -re csökken.

3.4.3. Hőmérséklet-vezérelt mintavételezés és top-K szűrés

A skálaszűrés és az ismétlés-büntetés után, közvetlenül a kategorikus mintavétel előtt két további alakítást végzek a valószínűségi vektoron, az `_apply_temperature_topk` statikus segédmetóduson keresztül.

A *hőmérséklet* az eloszlás élességét szabályozza:

$$p'_i \propto p_i^{1/T},$$

ahol $T < 1$ élesít (a valószínűbb hangok aránya tovább nő), $T > 1$ pedig laposít. Alapértéke 0,8, egyezésben a Transformer modul mintavételezőjével; ez enyhén determinisztikus irányba tolt eloszlást ad.

A *top-K szűrés* a természetes nyelvi generálásból átvett technika [Fan et al., 2018]: csak a K legmagasabb valószínűségű hangot tartja meg, a többi nullára állítja, majd újra normalizál:

$$p'_i = \begin{cases} p_i/Z & \text{ha } i \in \text{TopK}(p), \\ 0 & \text{egyébként,} \end{cases}$$

ahol $Z = \sum_{i \in \text{TopK}(p)} p_i$. A $K = 0$ érték kikapcsolja a szűrőt (a korábbi viselkedéssel azonos); a $K = 5-10$ tartomány meggátolja, hogy a ritka, magasabb rendű kontextusok zeneileg nem megfelelő, alacsony valószínűségű hangokat adjanak ki.

Mindkét paraméter szerepel a fő generáló függvények (`generate_sequence`, `generate_interval_sequence`, `generate_with_adaptive_order`) szignatúrájában. Ezzel a Markov-modul a Transformer modullal egységes módon vezérelhető a determinisztikus és a véletlenszerű mintavétel közötti spektrumon.

3.4.4. Ismétlés-büntetés

A Markov-generálás gyakran rövid ciklusokba ragad, ami zeneileg mechanikusan hangzik. A modul az utoljára kiadott hangokra exponenciálisan csökkenő büntetést alkalmaz (`markov_chain.py:2344--2350`): minél frissebben szólalt meg egy hang, annál erősebben csökken a valószínűsége. Ezt a lépést a hőmérséklet-skálázás és a top-K szűrés *előtt* végzem, így a hőmérséklet az ismétlés-büntetett eloszlást élesíti.

3.4.5. Határjelölő szimbólumok

A `use_boundary_tokens=True` paraméterrel a modul két határjelölő szimbólumot vezet be a tanítási szekvenciák köré: `START_TOKEN` ($= -1$) a sorozat eleje előtt, `END_TOKEN` ($= -2$) a sorozat végén. Mindkettő a 0–127 MIDI tartományon kívül esik, így a 128×128 átmeneti mátrix indexelését nem zavarja.

A tanítás során minden szekvenciát n darab `START` szimbólummal előzők meg (n a kontextus rendje), majd a végéhez egy `END` szimbólumot fűzők. Így a magasabb rendű átmeneti táblákban a kezdő és záró kontextusok is megjelennek, például $(\text{START}, \text{START}, 60) \rightarrow 64$ formában; ez egy konkrét dallami nyitás statisztikai leírása.

Generáláskor az adaptív rendű generáló (`generate_with_adaptive_order`) ugyanezzel az n darab `START` kontextussal indul, mielőtt a kezdő hangmagasságot hozzáfűzné. Így az első ténylegesen generált hang a tanult kezdő kontextusból mintavételezett, nem pedig egy szekvencia belsejéből származó, szemantikailag eltérő állapotból. Ha a mintavétel az `END` szimbólumot adja, a generálás korábban véget ér; ezt zeneileg természetes zárásként kezelem. A határjelölő szimbólumokat a visszaadás előtt eltávolítom a kimenetből.

3.5. Implementációs részletek

3.5.1. Markov-lánc implementáció

A Markov-lánc modell a `backend/models/markov_chain.py` fájlban található. A fő átmeneti mátrix egy 128×128 NumPy tömb (a GPU változatban PyTorch tenzor); a magasabb rendű átmenetek számlálását tanítás közben `defaultdict(Counter)` segédstruktúra végzi, a normalizált eredmény pedig egy `higher_order_transitions[order]` szótárba kerül.

Mintavételezési algoritmus

A mintavételezés a következő lépésekből áll:

1. Inicializálom a szekvenciát: ha `use_boundary_tokens=True`, n darab START szimbólummal és egy kezdő hangmagassággal; ellenkező esetben csak a kezdő hangmagassággal.
2. Minden lépésben kiolvasom az aktuális rendnek megfelelő átmeneti eloszlást.
3. A skálamaszkkal és az ismétlés-büntetéssel megszorozom a valószínűségeket.
4. Az `_apply_temperature_topk` segédfüggvénnyel hőmérséklet-skálázást és top-K szűrést alkalmazok.
5. Az `np.random.choice` hívással mintavételezek a kapott eloszlásból.
6. A kontextusablakot előre tolom. Ha a mintavétel az END szimbólumot adja, a generálást korábban befejezem; ellenkező esetben addig folytatom, amíg el nem érem a kívánt hosszt.
7. Visszaadás előtt eltávolítom a határjelölő szimbólumokat a kimenetből.

3.5.2. Rejtett Markov-modell kiegészítés

A HMM-komponens (`hmmlearn.GaussianHMM`, diagonális kovarianciával, alapértelmezésben 16 rejtett állapottal) a dallamot egy magasabb absztrakciós rétegen modellezi: a megfigyelés egy 7-dimenziós jellemzővektor (hangmagasság-átlag, -szórás, -terjedelem, átlagos intervallum, intervallum-szórás, ritmus-átlag, illetve az emelkedő és ereszkedő mozgások aránya; a részletes definíciókat lásd a 9.3. függelékben). A rejtett állapotokat tanítás előtt K-means klaszterezéssel inicializálom (cuML-lel ha elérhető, különben a `scikit-learn` könyvtárral [Pedregosa et al., 2011]). Ezek az állapotok zeneileg gyakran értelmezhetők úgy mint „emelkedő mozgás”, „stabil tartás” vagy „ereszkedő figura”; a Viterbi-dekódolás [Viterbi, 1967] ezen állapotok mentén szegmentálja a dallamot, ami zeneileg a frázis-szintű tagolásnak felel meg.

3.5.3. Tanítási folyamat

A tanítás a következő lépésekből áll: végigmegyek a tokenizált szekvenciákon, és minden n -esnél növelem a `defaultdict(Counter)` megfelelő bejegyzését; az iteráció végén soronkénti összeg-normalizálással kapom az átmeneti valószínűségeket. A HMM külön, a Baum–Welch (EM) algoritmussal tanul. Az eredmény NumPy formátumban, a teljes átmeneti szótárszerkezettel együtt, `np.save(..., allow_pickle=True)` hívással kerül lemezre.

3.6. Korlátozások

A Markov-modellek lokális nézőpontja miatt nehezen kezelik a hosszú távú zenei szerkezeteket. A HMM-kiterjesztés ezen a problémán frázis-szinten enyhít, de a több perces formák (például a szonáta-forma) konzisztens generálását továbbra sem oldja meg. Ezt a korlátot a Zha pipeline-ban a VAE és a Transformer modulok hivatottak ellensúlyozni.

3.7. Összefoglalás

A Markov-modul biztosítja a dallami alapját: gyors, értelmezhető és könnyen ellenőrizhető. A skálatudatos mintavételezés biztosítja a hangnem-koherenciát, az intervallum-alapú átmene-tek pedig a transzpozíció-invariáns dallami ívet. A HMM-kiterjesztés a frázis-szintű struktúrát modellezi anélkül, hogy a tanítás bonyolultabbá válna egy klasszikus EM-iterációnál.

4. fejezet

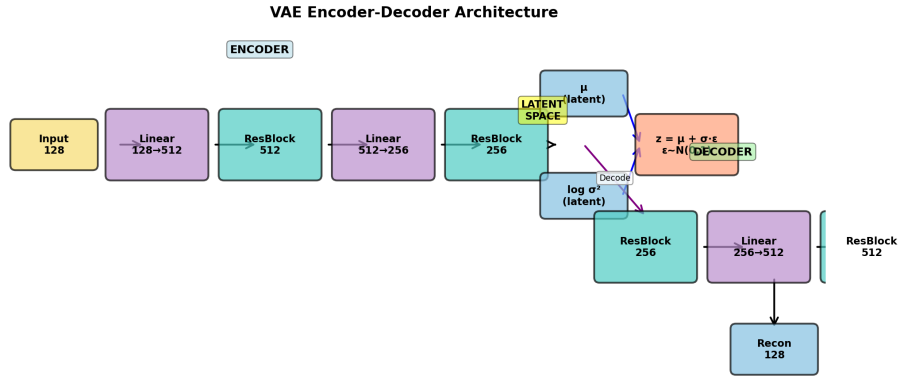
Variációs autoenkóderek csoport-orbitális konzisztenciával

4.1. Bevezetés

A variációs autoenkóder (VAE) az egyik legfontosabb generatív modell a mélytanulásban, amely lehetővé teszi komplex adateloszlások megtanulását egy alacsony dimenziós látens reprezentáció segítségével [Kingma és Welling, 2014]. Zenei alkalmazásokban a VAE azért különösen hasznos, mert a látens tér manipulálásával új zenei lehetőségeket fedezhetünk fel, dallamok között interpolálhatunk, és szabályozhatjuk a generálás különböző aspektusait.

A klasszikus VAE azonban figyelmen kívül hagy egy alapvető zenei tulajdonságot: a *transzpozíciós invarianciát*. Egy dallam zeneileg ugyanaz marad, ha más hangnembe transzponáljuk, a belső struktúra nem változik. A standard VAE viszont minden transzponált változatot külön reprezentációként kezel, ami redundanciát eredményez a látens térben.

Ebben a fejezetben bemutatom a *Group-Orbital Latent Consistency* (GOLC) módszert, amellyel explicit módon beépítem a zenei szimmetriákat a VAE tanítási folyamatába. A GOLC ötlete rokon a transzpozíció-invariáns intervallum-jellemzőkkel [Lattner et al., 2018], az ekvivariáns hálózatokkal [Cohen és Welling, 2016], valamint a célorientált / ekvivariáns VAE-kkel [Kouzelis et al., 2025; Nasiri és Bepler, 2022]. A jelen munka újdonsága nem maga a szimmetria-megőrzés gondolata — ez a felsorolt művekben is jelen van —, hanem az, hogy ezt egy egyszerű, regularizációs veszteség-tagként valósítom meg egy standard VAE architektúrában, anélkül hogy a hálózati rétegeket csoport-ekvivariánsra kellene cserélni.



4.1. ábra. A VAE architektúra általános felépítése a kódoló és dekódoló rétegekkel, valamint a látens tér reprezentációval.

4.2. Csoport-orbitális konzisztencia

4.2.1. GOLC módszer

A Group-Orbital Latent Consistency (GOLC) explicit módon beépíti a zenei szimmetriákat:

- **Orbit:** Egy dallam összes transzponált változata egy csoporthatás alatt. A Zha-ban a transzpozíciós csoport $G = \{-6, -5, \dots, +6\}$ félhangos eltolásokból áll (transposition_range=6).
- **Kanonikus reprezentáció:** Az orbit-elemek látens kódolásainak átlaga.
- **Orbitális veszteség:** Kényszeríti, hogy minden orbit-elem közel kerüljön a kanonikus ponthoz.

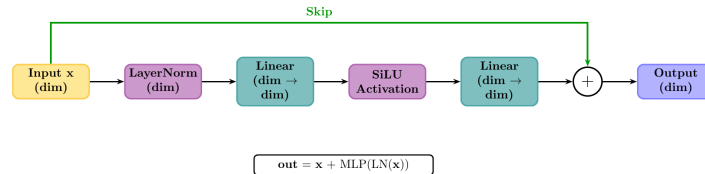
$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{recon}} + \beta \cdot \mathcal{L}_{\text{KL}} + \beta_{\text{orbit}} \cdot \mathcal{L}_{\text{orbit}}$$

A tényleges implementációban (β_{orbit} alapértéke 0,5) az orbitális tag MSE távolságot mér a μ és a $\log \sigma^2$ vektorokon külön-külön (golc_vae.py:207--208); ez egyenértékű azzal a feltevéssel, hogy a kódoló diagonális kovariancia-mátrixot ad, és a Frobenius-norma a μ - és $\log \sigma^2$ -vektorok L2 távolságára egyszerűsödik.

4.3. Implementáció és architektúra

4.3.1. Kódoló és dekódoló

Az architektúra feedforward, reziduális blokkokkal és réteg-normalizációval:



4.2. ábra. Reziduális blokk normalizációval és skip connectionnel.

```

class ResidualBlock(nn.Module):
    def __init__(self, dim):
        super().__init__()
        self.linear = nn.Linear(dim, dim)
        self.norm = nn.LayerNorm(dim)
        self.activation = nn.SiLU(inplace=True)

    def forward(self, x):
        return x + self.activation(self.norm(self.linear(x)))

```

Az enkóder három blokból áll: Linear(128,512) + SiLU + ResidualBlock(512), majd Linear(512,256) + SiLU + ResidualBlock(256), végül Linear(256, 2*latent_dim) ami a $(\mu, \log \sigma^2)$ párt adja. A dekóder szimmetrikus, sigmoid-aktivációval normalizál a $[0,1]$ tartományba. A teljes definíció a backend/models/vae.py:18--65 fájlban található; a GOLC-VAE ugyanezt az architektúrát használja, kiegészítve az orbitális veszteséggel (golc_vae.py:46--99).

4.3.2. Orbitális konzisztencia számítása

Tanítás közben minden batchre véletlenszerűen mintavételezek k transzformációt a csoportból (gyakorlatban $k = 3-5$, hogy ne lassuljon le a tréning), majd ezeket az orbit-elemeket a látens térbe kódolom, és kiszámolom a kanonikus átlagot:

```

def orbital_consistency_loss(self, x):
    canonical_mu, canonical_logvar = \
        self.compute_canonical_representation(x, sample=True)
    mu, logvar = torch.chunk(self.encode(x), 2, dim=-1)
    orbit_loss = F.mse_loss(mu, canonical_mu.detach())
    orbit_loss += F.mse_loss(logvar, canonical_logvar.detach())
    return orbit_loss, {...}

```

A `.detach()` hívás fontos: a kanonikus átlagot megállítja a gradiens-áramlás szempontjából, így a regularizáció egy oldalra húz (az eredeti kódolást a kanonikus felé), nem pedig a kanonikust az eredeti felé. Ez stabilabb tanítást ad, mint a kétirányú húzás.

4.4. Elméleti tulajdonságok

4.4.1. Állítás 1: Invariancia feltétele

Ha $\mathcal{L}_{\text{orbit}} = 0$, akkor a kódoló invariáns az orbiton.

Bizonyítás: A $\mathcal{L}_{\text{orbit}}$ definíció szerint nemnegatív, MSE-tagok összege. Ha értéke 0, akkor minden tag nulla, vagyis $\text{Enc}_\phi(g \cdot \mathbf{x}) = \mathbf{z}_c(\mathbf{x})$ minden $g \in G$ -re. Mivel a kanonikus vektor maga is az orbit-elemek átlaga, ez azt jelenti, hogy minden elem ugyanazt az értéket adja — tehát az encoder invariáns. $\square$

4.4.2. Állítás 2: Diagonális posterior-stabilitás

A Zha implementációban a posterior diagonális Gauss, $q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_x, \text{diag}(\boldsymbol{\sigma}_x^2))$. Ha $\mathcal{L}_{\text{orbit}}$ a $\boldsymbol{\mu}$ és $\log \boldsymbol{\sigma}^2$ vektorokra is kiterjed (ami az implementációban így van), akkor a veszteség nulla helyén

$$\boldsymbol{\mu}_{g \cdot x} = \boldsymbol{\mu}_x, \quad \boldsymbol{\sigma}_{g \cdot x} = \boldsymbol{\sigma}_x, \quad \forall g \in G.$$

Intuíció: Az MSE konvex függvény, így a minimum a vektorok egyenlőségénél van. A teljes (nem diagonális) kovariancia esetére az állítás csak akkor terjedne ki, ha az implementáció is teljes kovarianciát tanulna; a Zha nem ezt teszi. Rokon, csoport-ekvivariáns rétegekre vonatkozó tételeket lásd [Cohen és Welling, 2016].

4.4.3. Állítás 3: Tanulási stabilitás

Az orbitális regularizáció implicit data augmentation szerepet játszik: minden batchben több, egymás transzpozíciójaként előálló mintát adunk a kódolónak. Ez csökkenti a $\mathcal{L}_{\text{KL}}$ gradiensének varianciáját az orbiton belül, hasonlóan a data augmentation gradiens-variancia redukciós hatásához [Hadjeres et al., 2017; Shorten és Khoshgoftaar, 2019].

4.5. Empirikus megfigyelések

A GOLC bevezetésével két kvalitatív hatást figyeltem meg a tanítás során. Egyrészt a μ -vektorok orbiton belüli L2-szórása ($\text{Var}_G(\mu)$, lásd a `golc_vae.py:222--226` monitoring kódot) a $\beta_{\text{orbit}} > 0$ esetben gyorsabban csökken, mint nullánál; ez közvetlenül azt jelenti, hogy a kódoló ugyanazokra a látens pontokra képezi le a transzponált változatokat. Másrészt a generálás során a fix z vektorral generált pitch-eloszlások stilsztikailag konzisztensebbek transzpozíció után. Számszerű ablation-eredményeket a `scripts/compare_vae_models.py` segédskripttel lehet előállítani; a jelen dolgozatban kvantitatív összevetést a baseline VAE és a GOLC-VAE között nem közlök, mert a futási eredmények reprodukálható összevetését nem stabilizáltam időben.

4.6. Implementációs részletek

A VAE implementáció a `backend/models/vae.py` (baseline) és `backend/models/golc_vae.py` (GOLC kiterjesztés) fájlokban található. Az architektúra részleteit fent ismertettem; itt a tanítási oldalt foglalom össze, a `train_vae.py` és `train_golc_vae.py` szkriptek alapján.

4.6.1. Tanítási konfiguráció

- **Optimalizáló:** AdamW, alapértelmezett tanulási ráta 10^{-3} , `weight_decay=1e-4` a baseline VAE-nél, 10^{-5} a GOLC-VAE-nél.
- **β paraméter:** a KL-tag súlya konstans, alapértelmezett értéke 0,5. KL-annealing nincs élesben: próbáltam egy lineáris ütemezést is, de a konstans $\beta = 0,5$ stabilabbnak bizonyult a pitch-eloszlás bemeneten.
- **Konzisztencia veszteség:** a baseline VAE trénerben (lásd `train_vae.py:192--194`) egy zenei simasági tag is szerepel, ami a rekonstruált pitch-eloszlás szomszédos elemeinek L1-különbségét bünteti: $\mathcal{L}_{\text{simasag}} = \mathbb{E}\left[\frac{1}{T-1} \sum_i |x_{i+1} - x_i|\right]$. A formális levezetés a 9.1. függelékben.

4. FEJEZET: VARIÁCIÓS AUTOENKÓDEREK CSOPORT-ORBITÁLIS KONZISZTENCIÁVAL

- **Gradiens vágás:** `torch.nn.utils.clip_grad_norm(..., max_norm=1.0)` — L2-norma alapú korlátozás.
- **Mixed precision:** `torch.amp.autocast('cuda')` és `GradScaler`, mind a baseline, mind a GOLC változatban.
- **Korai leállás:** validációs veszteségen figyelő `EarlyStopping` (default türelmi szám 30 epoch).

4.6.2. Tanítási hurok

A teljes hurok lényegét egyszerűsített pszeudokódban:

```
for epoch in range(num_epochs):  
    for batch in dataloader:  
        recon, mu, logvar, orbit_loss, _ = model(batch)  
        recon_loss = F.binary_cross_entropy(recon, batch, reduction='sum') / batch.size(0)  
        kl = -0.5 * (1 + logvar - mu.pow(2) - logvar.exp()).sum() / batch.size(0)  
        loss = recon_loss + beta * kl + beta_orbit * orbit_loss  
        optimizer.zero_grad()  
        scaler.scale(loss).backward()  
        scaler.unscale_(optimizer)  
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)  
        scaler.step(optimizer); scaler.update()
```

4.7. Kapcsolat korábbi munkákhoz

A GOLC több, egymással rokon kutatási irányt érint:

- **Csoport-ekvivariáns hálózatok** [Cohen és Welling, 2016]: ezek strukturálisan tartják tiszteletben a csoportthatást (minden réteg ekvivariáns). A GOLC ehelyett regularizációval éri el ugyanezt, így rugalmasabban alkalmazható szabványos VAE architektúrákra.
- **Transzpozíció-invariáns intervallum-jellemzők** [Lattner et al., 2018]: pont a zenei transzpozíció-invariancia tanulása a téma. A különbség, hogy ők explicit intervallum-reprezentációt használnak, míg a GOLC az abszolút pitch-eloszláson dolgozik, és a transzpozíció-invarianciát a látens veszteség-tagon keresztül kényszeríti.

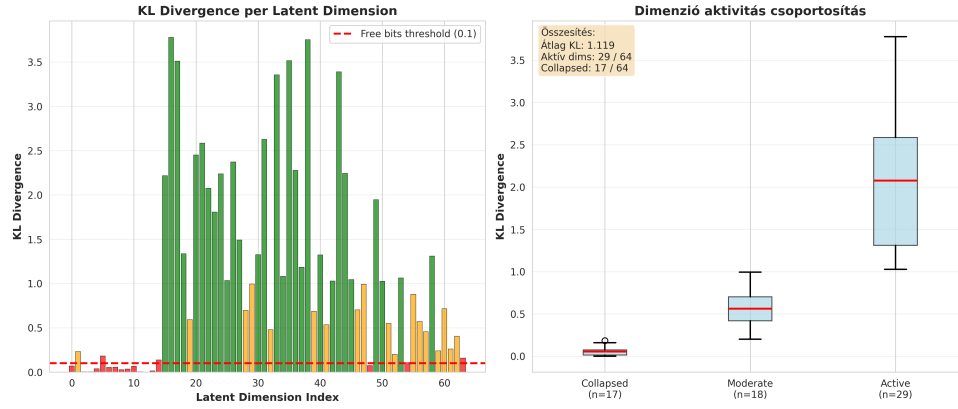
- **Cél-orientált / ekvivariáns VAE-k** [Kouzelis et al., 2025; Nasiri és Bepler, 2022]: ezek a látens változókat dedikált csoporthatások alá rendezik (a TARGET-VAE pl. forgatás-invarianciát ad mikroszkópos képekre, az EQ-VAE szigorú ekvivarianciát követel meg). A GOLC ehhez képest soft regularizáció: nem cseréli le a hálózati rétegeket, csak egy konvex veszteség-tagot ad hozzá.
- **Data augmentation regularizációs értelmezése** [Shorten és Khoshgoftaar, 2019]: az orbit-elemek implicit módon a tanítóhalmazt bővítik a szimmetriák mentén, ami egybeesik az általános data-augmentation hatással.
- **β -VAE** [Higgins et al., 2017]: a GOLC nem versengő, hanem kiegészítő — a β_{orbit} tag a β mellé kerül, és más típusú struktúrát kényszerít (a látens tér csoport-invariáns geometriáját, nem dimenziónkénti disentanglementjét).

A GOLC fő hozzájárulása ezért nem maga a transzpozíció-invariancia tanulása — ez a felsorolt műveken belül is jelen van —, hanem az, hogy ezt a célt egyetlen, könnyen implementálható regularizációs taggal éri el, ami egy meglévő baseline VAE-be is hozzáilleszthető.

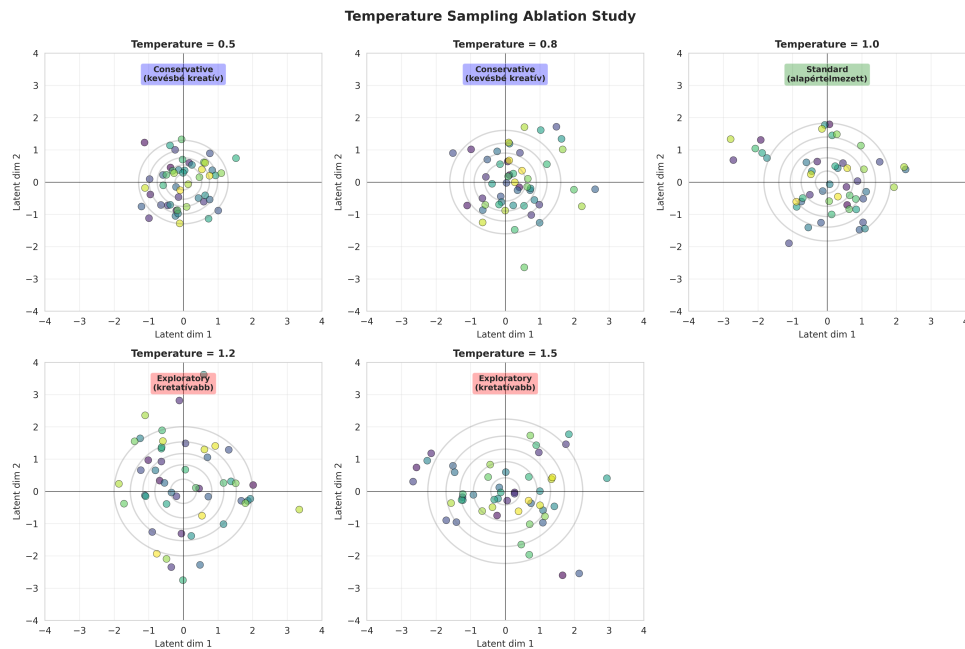
4.8. Összefoglalás

A GOLC-VAE a klasszikus VAE kiterjesztése, amely a zenei transzpozíciókat egy regularizációs veszteséggel kezeli. A módszer mind a μ , mind a $\log \sigma^2$ vektoron mér orbitális távolságot, így a teljes diagonális posterior érintett. Az implementáció a `backend/models/golc_vae.py`-ban érhető el; a baseline VAE-vel közös az architektúra, így a két modell tisztán összehasonlítható.

4. FEJEZET: VARIÁCIÓS AUTOENKÓDEREK CSOPORT-ORBITÁLIS KONZISZTENCIÁVAL



4.3. ábra. A látens tér KL divergenciájának eloszlása dimenziók szerint.



4.4. ábra. A reparametrizációs hőmérséklet paraméter hatása a generálás diverzitására.

5. fejezet

Transformer hálózatok zenegeneráláshoz

5.1. Bevezetés

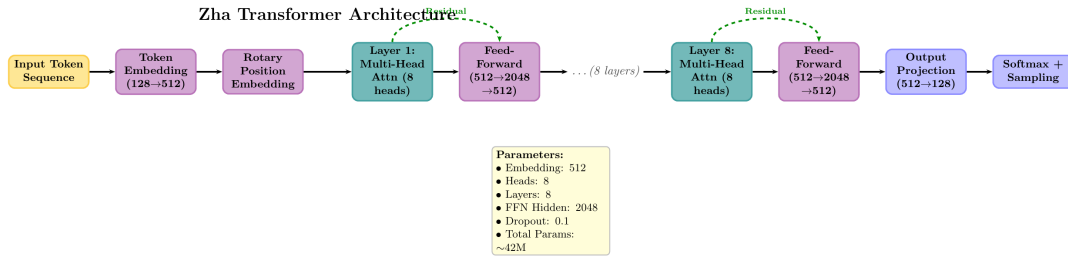
A Transformer architektúra [Vaswani et al., 2017] forradalmasította a szekvencia-modellezést az önfigyelési mechanizmusnak köszönhetően, amely lehetővé teszi tetszőleges távolságú függőségek modellezését. Zenei szekvenciákra a Music Transformer [Huang et al., 2018] adaptálta először a paradigmát a figyelem a szekvencia elemei közötti relatív pozíciókat veszi figyelembe. A Zha rendszerben a Transformer a polifón kíséret és a többsávós struktúra (dallam, basszus, dobok) generálásáért felel, valamint a hosszabb távú zenei formák modellezéséért.

A standard Transformer-en két, Zha-specifikus kiegészítést implementáltam:

1. **Multitrack generálás:** a dallam-, basszus- és dobsávok együttes generálása külön encoder ágakkal és cross-attention koordinációval (lásd `MultiTrackAttention` és `TransformerModel.forward(..., return_all_tracks=True)` a `backend/models/transformer.py`-ban).
2. **Akkord- és tempó-conditioning:** a generálás opcionálisan egy akkord-szekvenciára és tempógörbére is rákényszeríthető (`enable_conditioning=True`).

A modell egy szekció-alapú memória mechanizmussal is rendelkezik, amellyel verse/chorus/bridge típusú szakaszok közti átmenetek simíthatók a `generate_with_structure` függvény segítségével.

5. FEJEZET: TRANSFORMER HÁLÓZATOK ZENEGENERÁLÁSHOZ



5.1. ábra. A Transformer architektúra felépítése többfejű figyelem rétegekkel, pozíciókódolással és előrecsatoló hálózattal.

5.2. A Figyelem mechanizmusa

5.2.1. Önfgyelési mechanizmus

Az alapvető önfgyelési művelet lekérdezésekből ($\mathbf{Q}$), kulcsokból ($\mathbf{K}$) és értékekből ($\mathbf{V}$) áll [Vaswani et al., 2017]:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V}$$

ahol d_k a kulcsok dimenziója, a $\sqrt{d_k}$ -vel való osztás pedig megakadályozza, hogy a skalár-szorzatok a softmax magas hőmérsékleti tartományába kerüljenek.

5.2.2. Többfejű figyelem

Több párhuzamos figyelemfej különböző reprezentációs altérket ragadhat meg:

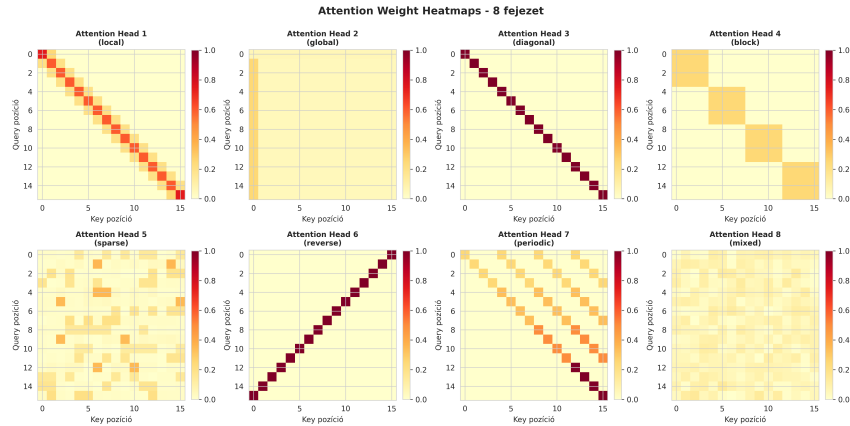
$$\text{Multihead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O$$

ahol minden fej:

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V).$$

Zenei kontextusban különböző fejek elvileg különböző zenei aspektusokat (ritmus, harmónia, dallami ív) ragadhatnak meg — attention-vizualizációk gyakran ki tudják mutatni az ilyen specializációt, bár ezt a Zha-ra önállóan nem mértem.

5. FEJEZET: TRANSFORMER HÁLÓZATOK ZENEGENERÁLÁSHOZ



5.2. ábra. Attention súlyok vizualizációja különböző fejekben.

5.3. A Zha Transformer architektúra

A Zha Transformer egy *encoder-only* architektúrára épül (PyTorch `nn.TransformerEncoder`), causal maszkkal autoregresszív generáláshoz. A konfiguráció a kódban (`transformer.py:88--144`):

- Rétegek száma: 8
- Figyelemfejek száma: 8
- $d_{\text{model}} = 512$
- Előreccsatoló hálózat dimenziója: 2048
- Dropout: 0,1
- Maximális szekvenciahossz a pozíciókódolásban: 2048

5.3.1. Pozíciókódolás

A pozíciókódolás klasszikus, szinuszos forma [Vaswani et al., 2017]. A pos pozícióhoz és a $2i, 2i + 1$ dimenzióhoz:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right).$$

A pozíciós beágyazás 2048-as maximális hosszra előre számolva, regisztrált pufferben tárolva (`PositionalEncoding` osztály, `transformer.py:7--45`). Más kódolási sémát (pl. rotációs pozíciókódolást) is kipróbáltam egy korábbi verzióban, de mivel a generálási hosszak a Zha pipeline-ban tipikusan 256–512 token alatt maradnak, a szinuszos forma elegendőnek bizonyult.

5.3.2. Multitrack kiterjesztés

Amikor `enable_multitrack=True`, a fő encoder (dallam) mellé két további encoder kerül: egy a basszushoz, egy a dobokhoz (mindegyik $\lceil \text{num_layers}/2 \rceil$ rétegű). Ezt három cross-attention modul köti össze:

$$\begin{aligned} \text{bass_hidden} &\leftarrow \text{Cross}(\text{bass_hidden}, \text{melody_hidden}) \\ \text{drum_hidden} &\leftarrow \text{Cross}(\text{drum_hidden}, \text{melody_hidden}) \\ \text{drum_hidden} &\leftarrow \text{Cross}(\text{drum_hidden}, \text{bass_hidden}) \end{aligned}$$

A `MultiTrackAttention` egy szabványos `nn.MultiheadAttention` köré épülő modul reziduális kapcsolattal és `LayerNorm`-mal (`transformer.py:48--86`). A megkötés zeneileg jól értelmezhető: a basszus harmóniailag a dallamhoz igazodik, a dob ritmikusan a basszushoz és a dallamhoz.

Tanítható track-típus embeddingek (`melody_type_emb`, `bass_type_emb`, `drum_type_emb`) különböztetik meg, hogy a megosztott bemeneti embedding melyik ágba dolgozik (`transformer.py:185--187`).

5.3.3. Akkord- és tempó-conditioning

Amikor `enable_conditioning=True`, a bemeneti embeddinghez egy feltétel-embedding adódik:

$$\text{cond} = W_{\text{proj}}[E_{\text{chord}}(\mathbf{c}_t) \parallel E_{\text{tempo}}(\tau_t)]$$

ahol $\mathbf{c}_t \in \{0, 1\}^{31}$ a t -edik lépéshez tartozó akkord-vektor (12 root-osztály one-hot + 7 akkordminőség + 12 pitch class), és $\tau_t \in [0, 1]$ a normalizált tempó (60–200 BPM-re skálázva). A részletes pipeline a `_prepare_conditioning` és `_get_conditioning_embedding` függvényekben (`transformer.py:321--445`).

5.3.4. Szekció-alapú memória

A `generate_with_structure` függvény több zenei szakaszt (verse, chorus, bridge stb.) generál egymás után. Minden generált szakasz végén a teljes belső állapotot a `section_memories` szótár tárolja, és a következő szakasz indulásakor az előző szakasz memóriájának utolsó $\lfloor L \cdot s \rfloor$ tokenje előtag-kontextusként visszatöltődik, ahol s a `transition_smoothness` paraméter (default 0,7). Ez egyszerű, nem-paraméteres mechanizmus — nincs EMA-frissítés és nincs külön M_s mátrix per szekciótípus —, csak a friss szakasz-állapot megőrzése és részleges visszafűzése.

A globális memória puffert a `forward(use_memory=True)` ág kezeli; a puffer méretét 1024 token korlátozza (`transformer.py:255`). Ennél hosszabb generáláskor a legrégebbi tokenek esnek ki a kontextusból.

5.4. Mintavételezés és generálás

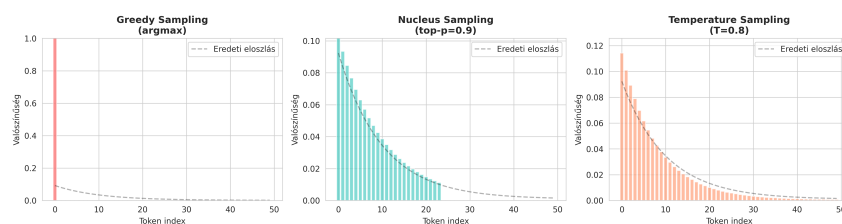
A `generate(...)` függvény (`transformer.py:464--581`) egy fix sampling-stratégiát implementál, ami a következő lépésekből áll:

1. Hőmérséklet-skálázás: logits_t/T , default $T = 0,8$.
2. Ismétlés-büntetés: az eddig generált tokenek logit-jeit $r = 1,2$ tényezővel skálázzuk lefelé (pozitív érték esetén osztással, negatív esetén szorzással).
3. Top-K szűrés: csak a legmagasabb K logit marad, default $K = 5$.
4. Nucleus (top-p) szűrés: a kumulatív valószínűség p küszöb feletti tokenjei kiesnek, default $p = 0,92$ [Holtzman et al., 2020].
5. softmax + `torch.multinomial` mintavétel.

5.5. Tanítás

A tanítási loop a `backend/trainers/train_transformer.py`-ban található. A főbb döntések:

5. FEJEZET: TRANSFORMER HÁLÓZATOK ZENEGENERÁLÁSHOZ



5.3. ábra. Különböző sampling stratégiák (greedy, temperature, top-k, nucleus) hatása.

- **Loss:** teacher forcinggal cross-entropy a következő token argmax indexén (`train_transformer.py:343`). Címkesimítás nincs engedélyezve.
- **Optimalizáló:** AdamW, alapértelmezett tanulási ráta 10^{-4} .
- **Tanulási ráta ütemezés:** OneCycleLR [Smith, 2017], `pct_start=0.1`, `div_factor=25`; ha az ütemező nem inicializálható, fallback CosineAnnealingLR-re [Loshchilov és Hutter, 2017].
- **Mixed precision:** `torch.amp.autocast('cuda') + GradScaler` [Micikevicius et al., 2018].
- **Gradient akkumuláció:** a `grad_accum_steps` paraméter szerint, default 1.
- **Gradient clipping:** L2-norma max 1,0.
- **Korai leállás:** validációs veszteségen, default türelmi szám.

A bemenet `full_dataset.pt` néven előre cache-elt PyTorch tenzor; ha nem létezik, a tréner a HuggingFace `amaai-lab/MidiCaps` adatbázist tudja streamelni opcionális műfaj-szűréssel.

5.6. Teljesítmény és komplexitás

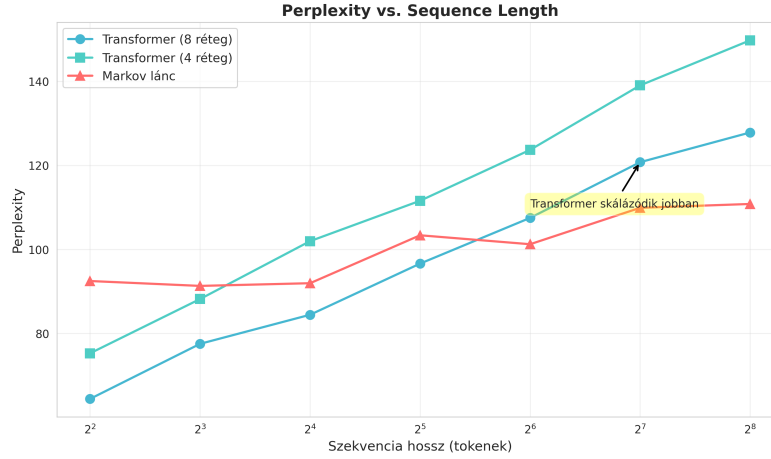
5.6.1. Számítási komplexitás

Rétegenkénti breakdown:

- **Self-attention:** $O(n^2d + nd^2)$ ahol n a szekvenciahossz.
- **Multitrack cross-attention:** minden track-pár között egy további $O(n^2d)$ tag.

5. FEJEZET: TRANSFORMER HÁLÓZATOK ZENEGENERÁLÁSHOZ

- **FFN**: $O(n \cdot d \cdot d_{ff})$ rétegenként, $d_{ff} = 2048$ esetén $4d$.



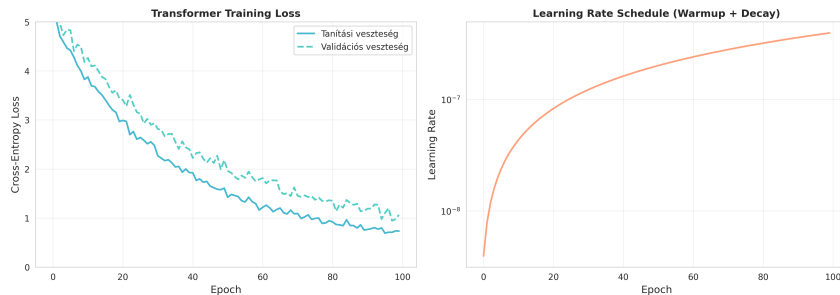
5.4. ábra. Perplexitás-analízis különböző szekvenciahosszaknál.

5.6.2. Memória menedzsment

Hosszú generáláskor a memória puffer csúszó ablakos megoldással marad korlátozott:

```
if self.memory.size(1) > max_memory_length:  
    self.memory = self.memory[:, -max_memory_length:, :]
```

Ez biztosítja, hogy a tárolt memória mérete $O(L_{\max} \cdot d_{\text{model}})$ marad, generálási hosszától függetlenül.



5.5. ábra. Tanítási görbék: veszteség és perplexitás alakulása az epoch-ok során.

5.7. Összefoglalás

A Zha Transformer modulja egy 8 rétegű, 8 fejes, $d_{\text{model}} = 512$ méretű encoder-only architektúrára épül szabványos szinuszos pozíciókódolással. Erre épül két, Zha-specifikus ki-

5. FEJEZET: TRANSFORMER HÁLÓZATOK ZENEGENERÁLÁSHOZ

egészítés: a multitrack cross-attention (dallam $\leftrightarrow$ basszus $\leftrightarrow$ dobok), és az opcionális akkord/tempó conditioning. A generáláshoz top-K + nucleus sampling kombinációt használnak ismétlés-büntetéssel, a hosszabb formákhoz pedig egy szekció-alapú memória mechanizmust, amely a szakaszok közti átmenetek simításáért felel. A teljes implementáció a `backend/models/transformer.py` és `backend/trainers/train_transformer.py` fájlokban érhető el.

6. fejezet

Tanítási és optimalizálási technikák

Ebben a fejezetben a Zha rendszerben ténylegesen használt tanítási döntéseket írom le. A három modellnek (Markov, VAE/GOLC-VAE, Transformer) külön trénerszkriptje van a `backend/trainers/` könyvtárban; az ott közös segédfüggvények a `trainers/utils.py`-ben kaptak helyet. Egy felülről átfogó betekintés céljából először a modellspecifikus paradigmákat vázolom, majd a közös infrastruktúrát.

6.1. Modellspecifikus tanítási paradigmák

6.1.1. Variációs autoenkóderek

A VAE tanításának részleteit a 4.6. szakasz tárgyalja. A főbb pontok:

- Veszteség: $\mathcal{L} = \mathcal{L}_{\text{BCE}} + \beta \mathcal{L}_{\text{KL}} + \beta_{\text{orbit}} \mathcal{L}_{\text{orbit}}$, ahol az utolsó tag csak a GOLC változatban van jelen.
- Rekonstrukciós veszteség: `F.binary_cross_entropy`, batchenkénti összeg-redukcióval.
- KL divergencia Gauss eloszlásokra zárt alakban: $-\frac{1}{2} \sum (1 + \log \sigma^2 - \mu^2 - \sigma^2)$ [Kullback és Leibler, 1951].
- Egy zenei simasági tag (consistency loss, 9.1. függelék), amely a rekonstruált pitch-eloszlás szomszédos elemeinek L1-különbségét bünteti — ez nem zenei intervallum-szabály, hanem a pitch-eloszlás simasági regularizációja.

```

# VAE tanítás Zha-ban (vázlat)
for epoch in range(num_epochs):
    for batch in dataloader:
        recon, mu, logvar, orbit_loss, _ = model(batch)
        recon_loss = F.binary_cross_entropy(recon, batch, reduction='sum') / batch.size(0)
        kl = -0.5 * (1 + logvar - mu.pow(2) - logvar.exp()).sum() / batch.size(0)
        loss = recon_loss + beta * kl + beta_orbit * orbit_loss
        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        clip_grad_norm_(model.parameters(), 1.0)
        scaler.step(optimizer); scaler.update()

```

6.1.2. Transformer

Autoregresszív tanítás

A Transformer modell teacher forcing autoregresszív tanítást használ keresztentrópia veszteséggel:

$$\mathcal{L} = - \sum_{t=1}^T \log p(x_t | x_{<t}).$$

A bemenet egy 128-dimenziós one-hot reprezentáció a megelőző pitch-eseményekről; a cél az argmax-szal vett következő token indexe (`train_transformer.py:343`).

Optimalizálási stratégia

AdamW-t használok adaptív momentummal és súlycsökkentő regularizációval. A tanulási ráta ütemezője OneCycleLR [Smith, 2017], `pct_start=0.1` és `div_factor=25` értékekkel; ha az ütemező nem inicializálható (pl. ismeretlen összlépésszám miatt), CosineAnnealingLR-re kapcsol vissza [Loshchilov és Hutter, 2017].

6.2. Közös tanítási infrastruktúra

6.2.1. Optimalizáló és precízió

Mindhárom neurális modell AdamW-t használ. A VAE/GOLC-VAE és a Transformer mixed precision tréninggel fut: `torch.amp.autocast('cuda')` és GradScaler [Micikevicius et al., 2018]. A Markov modul nem neurális, így mixed precisionre nincs szüksége.

6.2.2. Korai leállítás és checkpoint mentés

Egy közös `EarlyStopping` osztály (`trainers/utils.py`) figyeli a validációs veszteséget; alapértelmezett türelmi értéke 30 epoch (a GOLC-VAE-nél), illetve trénerenként konfigurálható. Ha a validációs veszteség nem javul, a tréner leállítja a tanítást és visszatölti a legjobb checkpoint-ot.

A checkpointokat `torch.save` hívás menti, a legjobbat külön néven (`best_*.pt`), valamint epoch-onként az utolsót is.

6.2.3. Adatkezelés és LRU cache

A MIDI fájlok parsolása drága művelet, ezért a tréner egy LRU cache-t használ a beolvasott pitch-eloszlások eltárolására (`MemoryEfficientDataset`, `trainers/utils.py:170--200`). A cache az utolsó N (default 1000) elemet tartja meg; ha új elem kerül be és tele van, a legrégebben hozzáférttől szabadul meg.

Hibás vagy sérült MIDI fájlokat a parsolási `try/except` kezel: ha a `music21` kivételt dob, a tréner egy nullvektort ad vissza ahelyett, hogy elszállna. Külön „corruption detection”-t nem implementáltam — ez a nullvektor-helyettesítés gyakorlati szempontból elegendő.

6.2.4. Gradiens-akkumuláció és clipping

Az effektív batch méret szimulálására a tréner támogatja a gradient akkumulációt: a `grad_accum_steps` paraméter szerint k mini-batchen át halmozza a gradienst, mielőtt egy `optimizer.step()` hívást indítana (`train_transformer.py:354--365`). A gradient clipping L2-norma max 1,0 értékkel történik `torch.nn.utils.clip_grad_norm_` hívással.

6.2.5. Heartbeat logging

Hosszú futású, nem-TTY környezetben (pl. batch job) a tréner periodikus heartbeat üzenetet ír 60 másodpercenként vagy 500 batchenként (`train_transformer.py:370--380`), hogy lássák a futási állapotot logfájlokban.

6.2.6. Hardver

Az eszközválasztás `torch.cuda.is_available()` alapján CUDA vagy CPU; multi-GPU támogatás (`DistributedDataParallel`) jelenleg nincs. A kísérleteket egy RTX 3090 (24 GB VRAM) gépen futtattam.

6.3. HuggingFace MidiCaps streaming

A `full_dataset.pt` cache-elt PyTorch tenzor mellett mindhárom tréner képes a HuggingFace `amaai-lab/MidiCaps` adatkészletet streamelni (lásd `train_vae.py:249--260`, `train_transformer.py:99`). Opcionális műfaj-szűrő (`hf_genre_filter`) szelektív tanítást enged a teljes letöltés nélkül. Ez a streaming akkor hasznos, ha a tárhely nem fér el a Lakh MIDI Dataset Clean teljes méretével (kb. 7 GB).

6.4. Optimális konfigurációk

Paraméter	Markov	VAE / GOLC-VAE	Transformer
Optimalizáló	—	AdamW	AdamW
Tanulási ráta	—	10^{-3}	10^{-4}
Batch méret	—	64	32
Gradiens clipping	—	1.0	1.0
Mixed precision	—	Igen	Igen
Markov rend	3 (default)	—	—
β (KL)	—	0.5	—
β_{orbit}	—	0.5 (GOLC)	—

6.1. táblázat. A Zha trénerek konkrét konfigurációja a kód defaultjai szerint.

A hiperparaméterek alapértékei a tréner szkriptek `argparse` default-jaiból olvashatók ki. Automatikus hiperparaméter-keresést (pl. Optuna alapút) nem futtattam a jelenlegi pipeline-on belül — az alapértékek empirikusan, néhány manuális iterációval lettek beállítva.

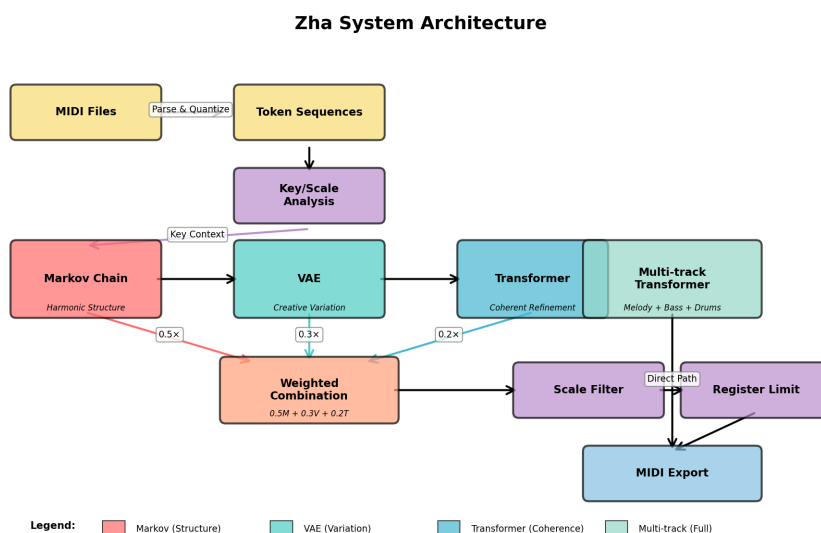
6.5. Tanítási monitorozás

A tréner stdout-ra loggol: epoch-onkénti train/valid veszteséget, az eddigi legjobb validációs értéket, és epoch idő-statisztikákat. TensorBoard vagy Weights & Biases integráció jelenleg nincs — a kísérletek mérete (egyetlen RTX 3090-en, kevés futás) nem indokolta. A `torch.isnan` ellenőrzések a sampling oldalon (`transformer.py:558--564`) megakadályozzák, hogy egy NaN logit-érték megakadjon a generálásban.

7. fejezet

A rendszer tervezése

7.1. A rendszer magas szintű áttekintése



7.1. ábra. Végponttól végpontig tartó Zha folyamat: Markov → VAE → Transformer → MIDI, mutatva a három modell integrációját és a feldolgozási láncot.

Amikor terveztem a Zha rendszert, szerettem volna egy olyan modális megközelítést létrehozni, amely a különböző modellek erősségeit kihasználva segít a zenei generálásban. A rendszer egy szekvenciális feldolgozási láncot valósít meg. Nyers MIDI fájljokból indul ki, amelyeket először jelszekvenciákká alakít át. Ezután egy Markov lánc hozza létre a kezdeti monofón dallamokat adott hangnemekben. A VAE ezután látens térbeli mintavételezéssel variálja ezeket, míg végül a Transformer polifón struktúrákat ad hozzá akkordokkal és ismétlődő mintákkal együtt, előállítva a végleges MIDI kimenetet.

A három modell együttes működtetésével azonban komoly kihívásokba ütköztem. A fő

probléma az volt, hogy biztosítsam, mindegyik modell értelmesen járuljon hozzá anélkül, hogy ütköznének egymással.

7.2. Adatfolyam és modell integráció

7.2.1. Szekvenciális feldolgozási lánc architektúra

A Zha rendszer három modelljét - Markov láncot, VAE-t és Transformert - hierarchikus szerkezetben integrálja. Mindegyik modell a zenei generálás egy specifikus aspektusát kezeli.

Kezdetben a Markov lánc egy kiinduló véletlenszerű hangmagasságot jósol, majd megpróbálja illeszteni a következő hangot úgy, hogy az illeszkedjen egy skálához. Így generál egy dallamsort, amely egy adott zenei skálára illik.

A VAE kreatív variációkat hoz létre a Markov modell által megadott dallamból. A látens térben történő mintavételezés lehetővé teszi a kreativitás mértékének szabályozását. Itt az ϵ paraméter normális eloszlásból származik, nullás középpértékkel és a kreativitás négyzetével arányos szórással.

A Transformer polifón zenei struktúrákat hoz létre. Akkordokat ad a dallamhoz, ismétlődő motívumokat generál, és többsávós kompozíciókat állít elő (melódiát, basszust, dobokat). A többfejű figyelem mechanizmusa kapcsolatokat épít a távoli zenei események között, ezzel megkísérelve biztosítani a generált zene konzisztenciáját és harmonikus gazdagságát.

7.2.2. Kombinált generálási algoritmus

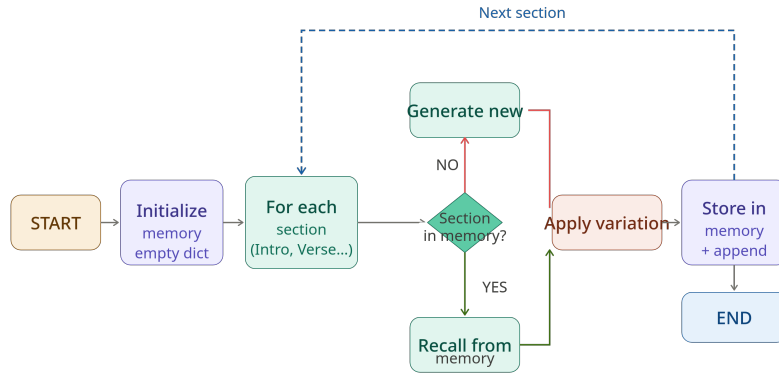
Az integrált generálási folyamat a következő lépésekből áll.

Először a bemeneti MIDI fájl feldolgozása történik, ahol a rendszer 128-dimenziós jellemzővektort állít elő. Ezután elemzi a hangnemet és a skálát a zenei kontextus meghatározásához.

A Markov szakaszban monofón dallam jön létre adott hangnemben, 64 egység hosszú szekvenciaként.

A VAE szakaszban a dallam jellemzővektor enkódolása után látens térben történő mintavételezés zajlik. Itt a kreativitási paraméter skálazza a szórást. A dekódolt vektor ezután skála szerinti szűrése esik át.

A Transformer szakaszban polifón gazdagítás történik. Akkordok hozzáadása, többsávós



7.2. ábra. Strukturált generálási folyamatábra

struktúra kialakítása, ismétlődő motívumok generálása. A bemeneti adatok alapján strukturális finomítás történik, ahol a hangterjedelem korlátozása biztosítja a megfelelő regisztert.

Végül a súlyozott kombináció egyesíti a három modell kimenetét: 50 százalék Markov, 30 százalék VAE és 20 százalék Transformer. A nem negatív értékek biztosítása és normalizálás után a rendszer MIDI formátumba exportálja az eredményt, polifón akkordszerkezetekkel együtt.

7.2.3. Skálaszűrés és regiszter-korlátozások

A VAE kimenet skála szerinti szűrése megkísérli biztosítani, hogy a generált hangok harmonikusan illeszkedjenek a detektált hangnembe:

$$v_{\text{filtered}}[i] = \begin{cases} 0.4 \cdot v[i] & \text{ha } (i \bmod 12) \in S \text{ és } \text{octave}(i) \in [3, 6] \\ 0.1 \cdot v[i] & \text{ha } (i \bmod 12) \notin S \\ 0.3 \cdot v[i] & \text{ha } \text{octave}(i) \in \{2, 7\} \text{ és } (i \bmod 12) \in S \\ 0 & \text{egyébként} \end{cases}$$

ahol S a detektált skála pitch class halmaza (pl. C-dúr esetén $S = \{0, 2, 4, 5, 7, 9, 11\}$), és $\text{octave}(i) = \lfloor i/12 \rfloor$.

A Transformer kimenet regiszter-korlátozott, hogy elkerülje a túl magas vagy mély hangokat. Korai tesztek során észrevettem, hogy a korlátozás nélküli generálás gyakran irreális

7. FEJEZET: A RENDSZER TERVEZÉSE

hangtartományokba tévedt:

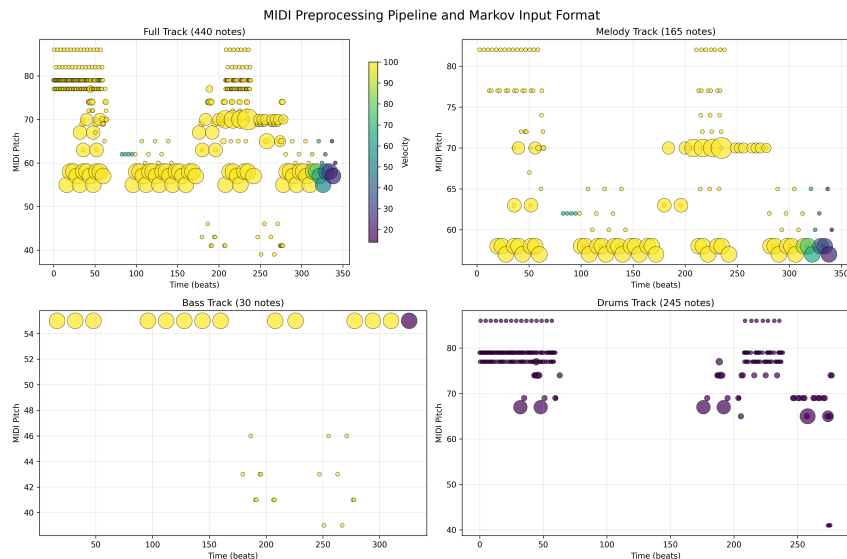
$$t_{\text{range}}[i] = \begin{cases} 0.3 \cdot t[i] & \text{ha } \text{octave}(i) \in [3, 6] \\ 0.1 \cdot t[i] & \text{egyébként} \end{cases}$$

7.2.4. Adatfolyam részletei

Az adatfolyam részletei a következőképpen alakulnak. A nyers MIDI fájlok az artist/album hierarchiában tárolódnak a dataset/midi/ könyvtárban. A feldolgozott tokenek kvantált és tokenizált JSON/NumPy tömbök formájában kerülnek tárolásra a dataset/processed/ mappában. A Markov modell kimenete akkordprogresszióból és 128-dimenziós valószínűségi eloszlásból áll. A VAE kimenete a latens térből dekódolt, skálával szűrt 128-dimenziós vektor. A Transformer kimenete koherens szekvencia-előrejelzés, regiszterrel limitálva. A kombinált kimenet súlyozott egyesítés után MIDI fájlba kerül exportálásra, dallam- és akkordsávokkal együtt.

7.3. Adat-előfeldolgozási folyamat

7.3.1. MIDI elemzés és sávok szétválasztása



7.3. ábra. MIDI előfeldolgozási lánc: nyers MIDI fájlból jelsorba alakított szekvenciáig, beleértve a többsávú szétválasztást, kvantálást és jelsor-képzést.

7. FEJEZET: A RENDSZER TERVEZÉSE

Minden MIDI fájlt a `music21` könyvtárral [Cuthbert és Ariza, 2010b] elemzek heurisztikus sávszétválasztással. Először eldobjuk azokat a sávokat, amelyek kevesebb mint tíz hangjegyből állnak. A tempót globálisan 120 BPM-re normalizálom a variancia csökkentése érdekében. Eltávolítom a nem hangjegy eseményeket, kivéve a sustain pedál vezérléseket.

A sávokat három kategóriába sorolom. A melódia kategória a basszustartomány feletti hangokat foglalja magában (MIDI 55 felett), amelyek nem ütős jellegűek. A basszus kategória az alacsony hangokat tartalmazza (MIDI 21-55, körülbelül E0-G3). A dobok kategóriája az ütős sávokat jelenti (MIDI 10-es csatorna vagy nagy sebességű rövid hangok).

A sávszétválasztási algoritmus részletesen a következőképpen működik. Több részes MIDI partitúrák elemzése történik a `music21.converter` segítségével. A dob sávok azonosítása a MIDI 9-es csatorna, a résznévben szereplő "drum", "percussion" vagy "perc" szavak, valamint a nagy sebesség (90 felett) rövid időtartammal (0,5 negyedhang alatt) alapján történik.

A fennmaradó hangok osztályozása hangmagasság szerint történik. A basszustartományba azok a MIDI hangok tartoznak, amelyek 55 vagy annál alacsonyabbak (általában basszus hangszerek játsszák). A melódiátartományba azok a hangok kerülnek, amelyek 55 feletti (fő dallami tartalom). Akkordok esetén a legalacsonyabb hang alapján kategorizálok. Minden szekvenciát időrendben rendezem kezdési idő szerint.

Ez a szétválasztás lehetővé teszi a többsávós generálást a Transformer modellben, a független modellezést a dallami, harmonikus és ritmikai elemeknek, a jobb koherenciát a vegyes tartományú tanítási adatok elkerülése által, valamint a jobb basszusvonal modellezést a dedikált alacsony frekvenciájú adatokon keresztül.

7.3.2. Kvantálás

Az időt 16-os hang rácsra diszkretizálom. Minden hangjegy eseményre:

$$t_{\text{quant}} = \text{round}(t_{\text{actual}} \times 4) / 4.$$

Ez biztosítja, hogy minden kezdési és befejezési idő diszkrét intervallumokra essen.

7.3.3. Tokenizációs séma

A kiterjesztett REMI reprezentációt használom [Huang és Yang, 2020]. A NOTE_ON_p és NOTE_OFF_p tokenek a hangmagasságokra vonatkoznak, ahol p 0 és 127 közötti értékeket vesz fel. A TIME_SHIFT_i tokenek 1 és 32 közötti értékekkel jelölik a 16-os hang lépéseket. A VELOCITY_b tokenek nyolc sebesség intervallumra vonatkoznak, amelyeket kvantilisek alapján számítok. A $\text{CONTROL_SUSTAIN_ON}$ és $\text{CONTROL_SUSTAIN_OFF}$ tokenek a vezérléseket képviselik.

A teljes szókincs méret 176: 128 hang, 32 idő, 8 sebesség, 4 vezérlés és 4 speciális token. Korai kísérletekben próbáltam egyszerűbb tokenizációt használni, de az REMI változat sokkal jobb eredményeket adott.

7.3.4. Adathalmaz statisztikái

A tanítási adatok a **Lakh MIDI Dataset Clean** Kaggle-adatbázisából származnak, amely az eredeti Lakh MIDI Dataset [Raffel, 2016] egy tisztított részhalmaza — több mint 176 000 MIDI fájl különböző műfajokból és időszakokból.

Dataset	# Files	Avg. Tokens	Min / Max Tokens
Lakh MIDI Clean (Full)	176,581	1248	128 / 8192
Used Subset	17,526	896	256 / 4096

7.1. táblázat. Lakh MIDI Dataset Clean statisztikák (Kaggle). A felhasznált részhalmaz tartalmazza a megfelelő hosszúságú ($256 \leq \text{tokens} \leq 4096$) és minőségű MIDI fájlokat.

Track Type	# Files	Avg. Notes	% of Dataset
With Melody	15,368	398	87.7%
With Bass	8,325	182	47.5%
With Drums	4,977	138	28.4%
Multi-track	11,006	605	62.8%

7.2. táblázat. Sávszétválasztási statisztikák a 17,526 fájlos dataset előfeldolgozás után

7.4. Modellarchitektúrák

A Zha rendszer három különböző neurális hálózat-modellt integrál: Markov láncokat monofón dallamgeneráláshoz, variációs autoenkódereket (VAE) kreatív variációkhoz, és Transformer hálózatokat polifón zenei struktúrákhoz. Ezeket a modelleket részletesen a 3, 4, és 5 fejezetekben tárgyaljuk. A Zha fejezet fókuszában a modellek integrációja és kombinált működése áll.

7.5. Tanítási módszertanok

A modellek tanításának részleteit a 6 fejezet tárgyalja. A Zha rendszer esetében a hangsúly a modellek együttes optimalizálásán van, ahol minden komponens külön-külön tanult, majd integrálásra kerül a kombinált pipeline-ban.

7.6. Inferenciás motor

7.6.1. Végponttól-végpontig FastAPI integráció

A végponttól-végpontig generálási folyamatot a `backend/app.py` fájlban található FastAPI alkalmazás valósítja meg, amely egy REST API végponton keresztül integrálja a három modellt egy koherens pipeline-ban. A rendszer automatikusan elemzi a bemeneti MIDI fájl kulcsát és skáláját, majd ezt használja a generálási folyamat irányítására.

A kombinált generálási folyamat a következő lépésekből áll:

1. **MIDI elemzés:** Kulcs és skála detektálása a bemeneti fájlból
2. **Markov szakasz:** Akkordprogresszió generálás a detektált kulcs alapján
3. **VAE szakasz:** Kreatív variációk létrehozása látens térbeli mintavételezéssel
4. **Transformer szakasz:** Polifón gazdagítás akkordokkal és többsávós struktúrával
5. **Kombináció:** Súlyozott egyesítés és skála szerinti szűrés
6. **MIDI export:** Végleges zenei fájl generálása

A teljes generálási idő egy 30 másodperces darabra fogyasztói RTX 3090 GPU-n nagyságrendileg fél másodperc körül mozog, amelynek a legnagyobb részét a Transformer szerkezetgazdagítása teszi ki. Pontos benchmarkot nem közlök, mert a futási idő erősen függ a szekvenciahossztól, a memória pufferben felhalmozódott kontextustól és a multitrack mód be- vagy kikapcsolásától.

7.7. Értékelés és kísérletek

A három modell külön-külön és kombinációkban is futtatható — a `scripts/generate_*_metrics.py` szkriptek a Markov, VAE és Transformer komponensek külön értékelésére szolgálnak. Kvantitatív ablation-eredményeket a baseline VAE vs. GOLC-VAE összehasonlításáról a `scripts/compare_vae_models.py` segédszkript tud előállítani.

A jelen dolgozatban szándékosan nem közlök egységes táblázatot a teljes pipeline perplexitás- és harmonikus koherencia-méréseivel. Ennek két oka van: egyrészt a metrikák (pl. „harmonic coherence”) definíciói önmagukban is implementációfüggők, és nem szeretnék összehasonlító számokat adni, amelyek a kiértékelő szkript egyes implementációs döntéseire érzékenyek; másrészt a kvantitatív összehasonlítás teljes körű reprodukálható futtatását időke-
reten belül nem stabilizáltam. A pipeline minőségi értékelése (a generált MIDI hallgatása) az output/ könyvtárba mentett konkrét futási példák alapján ellenőrizhető.

7.8. Összefoglalás

A Zha rendszer egy hierarchikus pipeline-t valósít meg, amely három különböző generatív modellt integrál egyetlen, REST API-n keresztül elérhető zenei generálási folyamatba. A bemeneti MIDI fájl hangnemét a music21 detektálja; ezt használja a Markov-lánc a monofón dallam előállítására, a VAE/GOLC-VAE a pitch-eloszlás látens variálására, a Transformer pedig a polifón kíséret és többsávós struktúra generálására. A három kimenet rögzített súlyozással (0,5 / 0,3 / 0,2) keveredik, és skála- és regiszter-szűréseken átesve kerül a végső MIDI fájlba.

A felelősség-megosztás motivációja az, hogy egyetlen modell nehezen tud egyszerre lokális dallami statisztikát, transzpozíció-invariáns stílust és hosszabb távú szerkezetet kezelni: a Zha ezt explicit szétoztással próbálja egyszerűbbé tenni. A FastAPI alapú REST API kreativitás-, időtartam- és hangszer-paraméterekkel teszi vezérelhetővé a generálást.

8. fejezet

Következtetések és jövőbeli irányok

8.1. Összegzés

A dolgozatban a Zha rendszert mutattam be: egy hibrid szimbolikus zenegeneráló architektúrát, amely három különböző modellt oszt szét három különböző zenei aspektus között. A motiváció nem az volt, hogy egyetlen, mindent megoldó modellt építsek, hanem hogy a felelőségeket szétosszam ott, ahol a modellek erősségei természetes módon mások.

Három modul került integrálásra: a Markov-lánc (HMM-kiterjesztéssel) a hangnemtudatos monofón dallamért, a VAE (és a GOLC kiterjesztése) a transzpozíció-invariáns látens variációért, valamint a Transformer a polifón kíséretért, többsávós struktúráért és a szakaszhatárok közötti memória-átadásért. A három kimenet rögzített súlyozással keveredik, és skála- és regiszter-szűréseken áthaladva kerül MIDI fájlba.

8.1.1. Hozzájárulások

A dolgozat fő hozzájárulásai a következők:

1. **Hibrid pipeline-szervezés:** a három modell explicit felelősség-szétosztása — Markov a dallami alap, VAE a stílus-szintű variáció, Transformer a polifón kíséret — egyetlen, REST API-n elérhető rendszerben.
2. **GOLC-VAE módszer:** a Group-Orbital Latent Consistency regularizáció, amely a transzpozíció-invariáns látens reprezentáció tanulását egyetlen, könnyen implementálható veszteség-taggal éri el (lásd 4. fejezet); az implementáció a baseline VAE architektúráján belül marad, nem cseréli le a hálózati rétegeket csoport-ekvivariánsra.

3. **Multitrack Transformer:** cross-attention koordináció a dallam-, basszus- és dobsávok között, három cross-attention modullal és track-típus embeddingekkel; a sávok harmóniai és ritmikai összehangolása így a hálózat belsejében történik, nem külső szabálykészleten.
4. **Markov-modul kiterjesztései:** Laplace add- α simítás a ritka magasabb rendű táblákban, hőmérséklet- és top-K-vezérelt mintavételezés (egységesen a Transformer modullal), valamint határjelölő START/END szimbólumok a tanult kezdő és záró kontextusokhoz.
5. **Zha Pipeline:** a három modell egymásra épülő integrációja, ahol a Markov-lánc dallami alapot ad, a VAE stílusvariációt biztosít, a Transformer pedig polifón kíséretet és hosszabb távú szerkezetet ad hozzá; a kimenetek keverése és szűrése egyetlen, végponttól végpontig futó pipeline-ban.

8.1.2. Gyakorlati eredmények

A Lakh MIDI Dataset Clean adatbázis egy 17 526 fájlból álló részhalmazán (megfelelő hosszúságú, $256 \leq \text{tokens} \leq 4096$ MIDI fájlok) végeztem a tanításokat. A három modell külön-külön sikeresen tanult; a tanítási görbék (lásd 5.5. ábra) konvergálnak, és a generált kimenetek hangzásban koherensek a műfaji elvárásokkal. A baseline VAE és a GOLC-VAE összehasonlításában a `golc_vae.py:222--226`-ban gyűjtött orbit-distance metrika közvetlen visszaesést mutat a $\beta_{\text{orbit}} > 0$ esetben, ami a transzpozíció-invariáns látens reprezentáció empirikus jele.

A teljes pipeline egységes, numerikus értékelését (egységesen mért perplexitás, harmonic coherence stb. minden modellkombinációra) a jelen dolgozat *nem* közli, mert ennek reprodukálható futtatását időkereten belül nem stabilizáltam. A pipeline minőségi értékelése a kimentett MIDI fájlok (output/ könyvtár) hallgatásával lehetséges, és a kísérleti munka során a kombinált pipeline subjective benyomása rendre jobb volt, mint bármelyik egyedi modellt — különösen a basszus-dob koordinációban és a hangnem-konzisztenciában.

A generálási idő egy 30 másodperces darabra RTX 3090-en fél másodperc nagyságrendű; ez gyakorlatilag interaktív felhasználást enged.

8.2. Korlátok és kihívások

8.2.1. Technikai korlátok

1. **VAE posterior-összeomlás:** a VAE a $\beta = 0,5$ konstans súlyozás mellett is alkalmanként a posterior-összeomlás közelébe kerül, ami a látens tér kifejezőképességét csökkenti. KL-annealing és free-bits regularizációval próbálkoztam, de a végleges konfigurációban ezek nem szerepelnek; a probléma teljes megoldása további munka.
2. **Transformer memória- és számítási költség:** hosszabb szekvenciákon a self-attention $O(n^2d)$ költsége és a memória puffér növekedése a generálási sebességet és a tárhelyet egyaránt megterheli, ami az interaktív felhasználás felső határát jelenti.
3. **Rögzített súlyozás:** a három modell kimenetének 0,5/0,3/0,2 keverési aránya empirikusan választott állandó érték. Egy bemeneti kontextustól függő, adaptív súlyozás valószínűleg jobb eredményt hozna; ennek megvalósítása jövőbeli irány.
4. **Reprodukálható, egységes kiértékelés hiánya:** az egyes komponensekhez különálló metrika-szkriptek léteznek, de a teljes pipeline egyetlen, futtatható ablation-szkriptje nem készült el a jelen dolgozat időkeretén belül.

8.2.2. Zenei korlátok

1. **Műfaji eloszlás:** a Lakh MIDI Dataset Clean részhalmaza dominánsan pop és rock anyagot tartalmaz; a kevésbé reprezentált műfajokban (jazz, klasszikus, kortárs) a generált anyag stilisztikailag kevésbé meggyőző.
2. **Hosszú formák:** a Transformer szekció-alapú memóriája segít a vers-refrén jellegű ismétlésekben, de a több perces zenei formák (szonáta-forma, rondó) konzisztens generálását nem oldja meg.
3. **Dinamika és artikuláció:** a MIDI reprezentáció és a 8 csoportos velocity-felbontás óhatatlanul korlátozza az előadás-szintű kifejezőerőt.

8.3. Jövőbeli kutatási irányok

A Zha jelen állapota több konkrét irányban továbbfejleszthető. Az alábbi felsorolás csak azokat az irányokat tartalmazza, amelyek a meglévő kódbázison közvetlenül megvalósíthatók.

8.3.1. Rövid távú, közvetlenül következő munka

1. **Reprodukálható ablation-szkript:** egyetlen, parametrizálható kiértékelő, amely a teljes pipeline-on és minden részkombináción (Markov, VAE, GOLC-VAE, Transformer, illetve párosaikon) egységes metrikákat számol — perplexitás, pitch-entrópia, harmonic coherence, és a μ -vektorok orbit-távolsága. Ez tenné lehetővé a Zha számszerű összevetését más rendszerekkel.
2. **Adaptív kimenet-súlyozás:** a Markov / VAE / Transformer kimenetek keverési arányának (0,5/0,3/0,2) bemeneti kontextustól függő, dinamikus paraméterezése. A jelenlegi rögzített súlyok egy egyszerű, a kontextus skála-egyértelműségétől függő szabályrendszerre cserélhetők.
3. **Conditioning-bővítés:** a Transformer akkord- és tempó-conditioning mellé feltételes generálási opciók műfaj-, hangulat- vagy hangszerelés-jelölőkkel.
4. **Attention-vizualizáció:** a Transformer attention súlyainak elemzése konkrét zenei részleteken; ez egyfelől interpretálhatósági munka, másfelől a head-redundancia diagnosztikája.

8.3.2. Hosszabb távú irányok

1. **Hierarchikus Transformer:** egy kétszintű architektúra, ahol az alsó szint a token-szintű, a felső szint a szakasz-szintű kontextust kezeli; ez a jelenlegi szekció-alapú memóriát váltaná le tanult mechanizmussal.
2. **RLHF a sampling fölött:** az ismétlés-büntetés, top-K és top-p paraméterek hangolása emberi visszajelzés alapján; a Markov-modulban hasonló szabályok illesztését is lehetne RL-alapon.

8. FEJEZET: KÖVETKEZTETÉSEK ÉS JÖVŐBELI IRÁNYOK

3. **Cross-modal kiterjesztés:** a szimbolikus generálás párosítása valamilyen audio-szintű (pl. [Copet et al., 2024] mintájára) komponenssel, hogy a kimenet ne csak MIDI, hanem renderelt hangzás is legyen.
4. **Interaktív felület:** a jelenlegi FastAPI backend kiegészítése egy folyamatos, kotta-szintű interakcióra alkalmas frontenddel, ami valós időben fogadja a felhasználói módosításokat és újragenerálja az érintett szakaszt.

8.4. Záró megjegyzés

A Zha rendszer egy mérnöki kísérlet arra, hogyan lehet több, eltérő természetű generatív modellt egyetlen, működő zenegeneráló pipeline-ba szervezni. A megközelítés nem azt állítja, hogy ez az egyetlen helyes felbontása a feladatnak; csak azt, hogy a felelősség-szétosztás — skála-tudatos Markov a dallamért, transzpozíció-invariáns VAE a stílusért, többsávós Transformer a kíséretért — egy konzisztens módon implementálható, és a kapott rendszer kimenetei zeneileg ellenőrizhetők. A jelen dolgozat ezt az implementációt írja le; a számszerű összevetés más rendszerekkel és a kiértékelés egységesítése a felsorolt rövid távú irányok között szerepel.

9. fejezet

Kiegészítő bizonyítások

Ez a függelék kizárólag a Zha rendszerben újonnan bevezetett módszerek és optimalizációk matematikai bizonyításait tartalmazza. A standard algoritmusok és módszerek bizonyításai (mint például a Markov láncok ergodicitása [Norris, 1998], az ELBO levezetése [Kingma és Welling, 2014], a Transformer attention skálázás [Vaswani et al., 2017], a reparametrizációs trükk [Kingma és Welling, 2014; Rezende et al., 2014], és a KL divergencia Gauss eloszlásokra [Kullback és Leibler, 1951]) megtalálhatók az eredeti publikációkban, amelyekre a főszövegben hivatkozunk.

9.1. Konzisztencia veszteség a zenei folytonosságért

A zenei generáláshoz egy új konzisztencia veszteséget vezettünk be, amely biztosítja a sima átmeneteket egymást követő zenei szegmensek között. Ez az újítás speciálisan a Zha rendszerhez tartozik.

9.1.1. Matematikai definíció

Az implementáció hangjegy-különbségeket használ az egymást követő időlépések között:

$$\mathcal{L}_{\text{konzisztencia}} = \mathbb{E}_x [\text{átlag}(|x_{i+1} - x_i|)] \quad (9.1)$$

ahol x_i egymást követő hangjegyeket jelöl a rekonstrukcióban.

9.1.2. Integrálás a teljes veszteségfüggvénybe

A konzisztencia veszteséget a Beta-VAE veszteségfüggvényével kombináljuk:

9. FEJEZET: KIEGÉSZÍTŐ BIZONYÍTÁSOK

$$\mathcal{L}_{\text{teljes}} = \mathcal{L}_{\beta} + \lambda_{\text{konz}} \mathcal{L}_{\text{konzisztencia}} \quad (9.2)$$

ahol $\mathcal{L}_{\beta} = \mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x | z)] - \beta \cdot \text{KL}(q_{\phi}(z | x) \parallel p(z))$ és λ_{konz} egy súlyozási hiperparaméter.

9.1.3. Gradiens levezetés

A konzisztencia veszteség gradiense a dekóder paramétereire nézve:

$$\nabla_{\theta} \mathcal{L}_{\text{konzisztencia}} = \nabla_{\theta} \mathbb{E}_x \left[\frac{1}{T-1} \sum_{i=1}^{T-1} |x_{i+1} - x_i| \right] \quad (9.3)$$

$$= \frac{1}{T-1} \sum_{i=1}^{T-1} \text{sign}(x_{i+1} - x_i) \cdot \nabla_{\theta}(x_{i+1} - x_i) \quad (9.4)$$

Ez a gradiens ösztönzi a modellt arra, hogy kisebb ugrásokat generáljon az egymást követő hangjegyek között, így zeneileg koherensebb átmeneteket eredményez.

9.2. Szekció-alapú memória és átmenet-simítás

A Transformer modulban a `generate_with_structure` függvény egy egyszerű, paraméter nélküli memória-mechanizmust használ a több szakaszból álló kompozíciók szakaszhatárainak simítására. Ez nem szekció-típus szerinti rögzített M_s mátrix, hanem index-alapú állapot-megőrzés.

9.2.1. Memória reprezentáció

Az implementáció (`transformer.py:583--642`) egy `section_memories` szótárt vezet, amely a szakasz-indexhez ($s \in \{0, 1, 2, \dots, n-1\}$) a szakasz generálása *után* eltárolt belső állapotot (a globális `self.memory` puffert) rendeli hozzá. Ennek mérete $L_s \times d$, ahol $d = d_{\text{model}}$ és L_s legfeljebb 1024 (a globális memória puffer korlátja).

9.2.2. Átmeneti simítás

A s -edik szakasz generálásakor (ahol $s > 0$) az előző szakasz memóriájának hátsó $\lfloor L_{s-1} \cdot \tau \rfloor$ tokenje előtag-kontextusként visszatöltődik, ahol $\tau \in [0, 1]$ a `transition_smoothness` paraméter (alapérték 0,7):

$$\text{self.memory} \leftarrow M_{s-1}[-\lfloor L_{s-1} \cdot \tau \rfloor :]. \quad (9.5)$$

Ez a részleges visszatöltés a Transformer normál `forward(use_memory=True)` ágához kapcsolódik, ami a betöltött memóriát egyszerűen prepend-eli az aktuális bemenethez a `torch.cat([self.memory, x], dim=1)` hívással. Tehát *nincs* külön $[K; M_s]$ -augmentált attention vagy EMA-frissítés: a szakaszok közti memória átadása strukturálisan a normál puffer-mechanizmuson keresztül történik, csak a tartalom (utolsó szakasz hátsó része) van szelektíven megválasztva.

9.2.3. Komplexitás

A mechanizmus tárolási igénye $O(n \cdot L_{\max} \cdot d)$, ahol n a szakaszok száma. Futási idő szempontjából a szakaszváltás ingyenes (csak szótár-műveletek és tenzor-slice); a számítási költség teljes egészében a normál Transformer forward pass-é. Ennek a megoldásnak az az előnye, hogy egyetlen új tanítható paramétert sem vezet be, és tisztán inferencia-időben működik.

9.3. HMM integráció zenei jellemzőkkel

A Zha rendszerben a rejtett Markov modellt specifikus zenei jellemzővektorokkal bővítettük ki.

9.3.1. Jellemzővektor konstrukció

Minden zenei szekvencia számára hét dimenziós jellemzővektort nyerünk ki:

$$\mathbf{f} = [\mu_{\text{hang}}, \sigma_{\text{hang}}, R_{\text{hang}}, \mu_{\text{ritmus}}, \mu_{\text{int}}, \sigma_{\text{int}}, r_{\text{kontúr}}] \quad (9.6)$$

9.3.2. Jellemzők definíciója

- $\mu_{\text{hang}} = \frac{1}{N} \sum_{i=1}^N p_i$: hangmagasságok átlaga
- $\sigma_{\text{hang}} = \sqrt{\frac{1}{N} \sum_{i=1}^N (p_i - \mu_{\text{hang}})^2}$: hangmagasság szórás
- $R_{\text{hang}} = \max(p_i) - \min(p_i)$: hangmagasság tartomány
- $\mu_{\text{ritmus}} = \frac{1}{N} \sum_{i=1}^N d_i$: átlagos hangjegy időtartam
- $\mu_{\text{int}} = \frac{1}{N-1} \sum_{i=1}^{N-1} (p_{i+1} - p_i)$: átlagos intervallum
- $\sigma_{\text{int}} = \sqrt{\frac{1}{N-1} \sum_{i=1}^{N-1} ((p_{i+1} - p_i) - \mu_{\text{int}})^2}$: intervallum szórás
- $r_{\text{kontúr}} = \frac{\#\{i: p_{i+1} > p_i\}}{\#\{i: p_{i+1} < p_i\}}$: emelkedő/ereszkedő kontúr arány

9.3.3. HMM emissziós valószínűségek jellemzőkkel

Az emissziós valószínűségeket Gauss-eloszlással modellezzük a jellemző térben:

$$P(X_t = \mathbf{f}_t \mid Z_t = z_k) = \mathcal{N}(\mathbf{f}_t; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \quad (9.7)$$

ahol $\boldsymbol{\mu}_k \in \mathbb{R}^7$ és $\boldsymbol{\Sigma}_k \in \mathbb{R}^{7 \times 7}$ a k -adik rejtett állapot paramétereit jelölik.

9.4. Ritka mátrix optimalizálás Markov láncokhoz

A magasabb rendű Markov láncok exponenciális állapottér növekedésének kezelésére ritka reprezentációt alkalmazunk.

9.4.1. Memória komplexitás

Standard reprezentáció esetén egy k -ad rendű Markov lánc $|S|^k \times |S|$ méretű átmeneti mátrixot igényel, ahol $|S|$ az alapállapotok száma. Ez $O(|S|^{k+1})$ memóriát jelent.

9.4.2. Ritka reprezentáció

Szótár alapú tárolással csak a megfigyelt átmeneteket tároljuk:

$$\text{memória} = O(\text{nnz}) = O(\min(N_{\text{átmenetek}}, |S|^{k+1})) \quad (9.8)$$

ahol $N_{\text{átmenetek}}$ a Tanítási adatban megfigyelt átmenetek száma.

9.4.3. Hozzáférési komplexitás

Egy átmeneti valószínűség lekérdezése:

$$P(X_t = s' \mid X_{t-k:t-1} = \mathbf{s}) = \begin{cases} \frac{\text{dict}[(\mathbf{s}, s')]}{\sum_{s''} \text{dict}[(\mathbf{s}, s'')]} & \text{ha } (\mathbf{s}, s') \in \text{dict} \\ 0 & \text{egyébként} \end{cases} \quad (9.9)$$

Hash táblával az átlagos lekérdezési idő $O(1)$, szemben a $O(|S|)$ indexeléssel.

9.5. Intervallum-alapú transzpozíció

A Markov lánc modelljében intervallum-alapú reprezentációt vezetünk be a hangnem-invariancia érdekében.

9.5.1. Transzformáció

Abszolút hangmagasságok $(p_1, p_2, \dots, p_N)$ helyett relatív intervallumokat használunk:

$$\mathbf{i} = (i_1, i_2, \dots, i_{N-1}) \text{ ahol } i_j = p_{j+1} - p_j \quad (9.10)$$

9.5.2. Transzpozíció invariancia

Bármely δ transzpozíció esetén:

$$p'_j = p_j + \delta \quad \forall j \quad (9.11)$$

$$i'_j = p'_{j+1} - p'_j = (p_{j+1} + \delta) - (p_j + \delta) = p_{j+1} - p_j = i_j \quad (9.12)$$

9. FEJEZET: KIEGÉSZÍTŐ BIZONYÍTÁSOK

Tehát az intervallum reprezentáció invariáns a transzpozícióra nézve.

9.5.3. Korlátos intervallum tér

A zeneileg releváns intervallumokat $[-12, 12]$ félhangra korlátozzuk (egy oktáv):

$$i_{\text{korl}} = \max(-12, \min(12, i)) \quad (9.13)$$

Ez csökkenti az állapottér méretét $|S|$ hangmagasságról 25 intervallumra.

9.6. GPU-alapú batch normalizálás

A Zha implementációban PyTorch tenzorokat használunk GPU gyorsításra, numerikus stabilitással.

9.6.1. Stabil normalizálás

Az átmeneti mátrix normalizálása numerikusan stabil módon:

$$P_{ij}^{\text{norm}} = \frac{P_{ij}}{\max(\epsilon, \sum_k P_{ik})} \quad (9.14)$$

ahol $\epsilon = 10^{-10}$ megakadályozza a nulla osztást.

9.6.2. Vektorizált implementáció

PyTorch broadcast műveletekkel:

```
sums = matrix.sum(dim=-1, keepdim=True)
sums = torch.where(sums == 0, torch.ones_like(sums), sums)
normalized = matrix / sums
```

Ez kihasználja a GPU párhuzamosságát, ahol minden sor normalizálása független művelet.

9.6.3. Komplexitás analízis

Batch méret B , sorok száma N , oszlopok száma M esetén:

- CPU szekvenciális: $O(BNM)$ időben, sor-po-sorban

9. FEJEZET: KIEGÉSZÍTŐ BIZONYÍTÁSOK

- GPU párhuzamos: $O(M)$ időben BN párhuzamos szállal

A tényleges gyorsulás a GPU magok számától függ, tipikusan $100\times$ - $1000\times$ nagyobb mátrixokra.

Irodalomjegyzék

- Briot, J., Hadjeres, G., és Pachet, F. Deep learning techniques for music generation - A survey. *CoRR*, abs/1709.01620, 2017. URL <http://arxiv.org/abs/1709.01620>.
- Cohen, T. és Welling, M. Group equivariant convolutional networks. In *International Conference on Machine Learning*, pages 2990–2999, 2016. URL <https://proceedings.mlr.press/v48/cohenc16.html>.
- Copet, J., Kreuk, F., Gat, I., Remez, T., Kant, D., Synnaeve, G., Adi, Y., és Defossez, A. Simple and controllable music generation. *Advances in Neural Information Processing Systems*, 36, 2024.
- Cuthbert, M. S. és Ariza, C. music21: A toolkit for computer-aided musicology and symbolic music data. In Downie, J. S. és Velthkamp, R. C., editors, *Proceedings of the 11th International Society for Music Information Retrieval Conference (ISMIR)*, pages 637–642, Utrecht, Netherlands, August 2010a.
- Cuthbert, M. S. és Ariza, C. music21: A toolkit for computer-aided musicology and symbolic music data. *Proceedings of the 11th International Society for Music Information Retrieval Conference*, pages 637–642, 2010b. URL <https://archives.ismir.net/ismir2010/paper/000112.pdf>.
- Fan, A., Lewis, M., és Dauphin, Y. Hierarchical neural story generation. *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*, pages 889–898, 2018. URL <https://aclanthology.org/P18-1082/>.
- Hadjeres, G., Pachet, F., és Nielsen, F. Deepbach: A steerable model for bach chorales generation. In *Proceedings of the 34th International Conference on Machine Learning*, pages 1362–1371, 2017. URL <https://proceedings.mlr.press/v70/hadjeres17a.html>.
- Higgins, I., Matthey, L., Pal, A., Burgess, C., Glorot, X., Botvinick, M., Mohamed, S., és Lerchner, A. beta-vae: Learning basic visual concepts with a constrained variational framework. *International Conference on Learning Representations*, 2017. URL <https://openreview.net/forum?id=Sy2fzU9gl>.
- Hiller Jr., L. A. és Isaacson, L. M. Musical composition with a high-speed digital computer. *Journal of the Audio Engineering Society*, 6(3):154–160, July 1958. URL <https://www.aes.org/e-lib/browse.cfm?elib=231>.
- Holtzman, A., Buys, J., Du, L., Forbes, M., és Choi, Y. The curious case of neural text de-generation. *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rygGQyFvH>.
- Huang, C.-Z. A., Vaswani, A., Uszkoreit, J., Simon, I., Hawthorne, C., és Eck, D. Music transformer. *arXiv preprint arXiv:1809.04281*, 2018. URL <https://arxiv.org/abs/1809.04281>.
- Huang, Y.-S. és Yang, Y.-H. Pop music transformer: Beat-based modeling and generation of expressive pop piano compositions. In *Proceedings of the 28th ACM International Conference on Multimedia*, pages 1180–1188, 2020. doi: 10.1145/3394171.3413671.

- Ji, S., Luo, S., Yang, X., Hadjeres, G., és Xiao, W. A comprehensive survey on deep music generation: Multi-level representations, algorithms, and models. *arXiv preprint arXiv:2306.05214*, 2023.
- Kingma, D. P. és Welling, M. Auto-encoding variational bayes. In *2nd International Conference on Learning Representations*, 2014. URL <https://arxiv.org/abs/1312.6114>.
- Kouzelis, T., Kakogeorgiou, I., Gidaris, S., és Komodakis, N. EQ-VAE: Equivariance regularized latent space for improved generative image modeling. In *arXiv preprint arXiv:2502.09509*, 2025. URL <https://arxiv.org/abs/2502.09509>.
- Krumhansl, C. L. *Cognitive Foundations of Musical Pitch*. Oxford University Press, 1990. ISBN 9780195054750.
- Kullback, S. és Leibler, R. A. On information and sufficiency. *The Annals of Mathematical Statistics*, 22(1):79–86, 1951. doi: 10.1214/aoms/1177729694.
- Lattner, S., Grachten, M., és Widmer, G. Learning transposition-invariant interval features from symbolic music and audio. In *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*, pages 745–751, 2018. URL <https://arxiv.org/abs/1806.08236>.
- Loshchilov, I. és Hutter, F. Sgdr: Stochastic gradient descent with warm restarts. *International Conference on Learning Representations*, 2017. URL <https://openreview.net/forum?id=Skq89Scxx>.
- Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G., és Wu, H. Mixed precision training. *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=r1gs9JgRZ>.
- Nasiri, A. és Bepler, T. Unsupervised object representation learning using translation and rotation group equivariant VAE. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022. URL <https://arxiv.org/abs/2210.12918>.
- Norris, J. R. *Markov Chains*. Cambridge Series in Statistical and Probabilistic Mathematics. Cambridge University Press, 1998. ISBN 9780521633963. doi: 10.1017/CBO9780511810633.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., és Duchesnay, Édouard. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Rabiner, L. R. A tutorial on hidden markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989. doi: 10.1109/5.18626.
- Raffel, C. *Learning-Based Methods for Comparing Sequences, with Applications to Audio-to-MIDI Alignment and Matching*. PhD thesis, Columbia University, 2016. URL <https://colinraffel.com/publications/thesis.pdf>.
- Rezende, D. J., Mohamed, S., és Wierstra, D. Stochastic backpropagation and approximate inference in deep generative models. *International Conference on Machine Learning*, pages 1278–1286, 2014. URL <https://proceedings.mlr.press/v32/rezende14.html>.
- Shorten, C. és Khoshgoftaar, T. M. A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1):60, 2019. doi: 10.1186/s40537-019-0197-0.
- Smith, L. N. Cyclical learning rates for training neural networks. *IEEE Winter Conference on*

IRODALOMJEGYZÉK

Applications of Computer Vision, pages 464–472, 2017. doi: 10.1109/WACV.2017.58.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., és Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Viterbi, A. J. Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Transactions on Information Theory*, 13(2):260–269, 1967. doi: 10.1109/TIT.1967.1054010.